Del rastrojo al kilovatio-hora

Sistema de información para estimar el potencial eléctrico y económico de la biomasa residual agrícola de la Argentina a escala departamental

OPTIMIA-HV

Trabajo Final de Grado presentado para obtener el título de **Ingeniero en Sistemas de Información**

Autor	Marco Ferro
Institución	Universidad Nacional de Villa Mercedes (UNVM)
Carrera	Ingeniería en Sistemas de Información
Director/a	*[a completar]*
Lugar y fecha	Argentina, noviembre de 2026
Repositorio	`github.com/marcoeferro/SYNGAS-OPTIMIA-HV`

Resumen

La Argentina produce cada año más de 140 millones de toneladas de granos y, con ellos, una cantidad comparable de residuos de cosecha (rastrojo) que permanecen en el lote. Este trabajo final de grado presenta el diseño, la construcción y la puesta en producción de **OPTIMIA-HV**, un sistema de información que estima, para cada departamento del país, cuánta biomasa residual dejan los cultivos, cuánta electricidad podría generarse con ella mediante un gasificador comercial de pequeña escala (All Power Labs PP30, 22 kW) y cuánto dinero representa esa electricidad bajo el régimen de generación distribuida de la Ley 27.424, a las tarifas vigentes de cada distribuidora.

El sistema se compone de un proceso ETL (*Extract, Transform, Load*) escrito en Python que descarga de manera automática y semanal las estimaciones agrícolas del Ministerio de Agricultura, Ganadería y Pesca (MAGyP) y de la Bolsa de Cereales de Córdoba, y los cuadros tarifarios de las 28 distribuidoras eléctricas del país, publicados en formatos heterogéneos (PDF, planillas, páginas web dinámicas y API). Los datos se normalizan, se reconcilian entre fuentes, se transforman mediante un modelo físico-económico lineal y auditable, se someten a invariantes y alertas de calidad, y se publican en archivos Parquet que consume un tablero web interactivo construido con Streamlit. Toda la solución se empaqueta en contenedores Docker, se programa con un planificador tipo *cron*, se valida mediante integración continua en GitHub Actions y se opera en un servidor privado virtual (VPS) administrado por SSH.

Para el período de referencia 2021/22–2025/26, el modelo estima un potencial técnico de **160,2 millones de toneladas de materia seca de rastrojo por año**, de las cuales 80,1 Mt se consideran retirables sin comprometer el suelo; ello equivale a **80,1 TWh/año** de electricidad (del orden del 58 % de la demanda eléctrica nacional de 2022) y a un **beneficio bruto de 11,2 billones de pesos por año**. El 74,8 % del potencial se concentra en Buenos Aires, Córdoba y Santa Fe, y el 84,6 % proviene de maíz, soja y trigo. El supuesto más influyente es la fracción de rastrojo retirada. Más allá de las cifras, el aporte principal del trabajo es una infraestructura de datos reproducible, trazable y tolerante a fallos, que convierte información pública dispersa en un insumo actualizado para la planificación energética.

Palabras clave: biomasa residual agrícola; gasificación; generación distribuida; ETL; extracción web; ingeniería de datos; Docker; integración continua; SSH; tablero de control.

Abstract

Argentina harvests over 140 million tonnes of grain every year, leaving a comparable amount of crop residue on the fields. This final degree project presents the design, construction and deployment of **OPTIMIA-HV**, an information system that estimates, for every department of the country, the residual biomass left by crops, the electricity that could be generated from it with a small commercial gasifier (All Power Labs PP30, 22 kW), and its monetary value under Argentina's distributed generation law (Law 27,424) at each utility's current tariffs. The system is built around a weekly, fully automated Python ETL pipeline that scrapes agricultural statistics and the tariff schedules of the country's 28 electricity distributors from heterogeneous sources (PDF, spreadsheets, dynamic web pages and APIs), reconciles and validates the data, applies a linear and auditable physical-economic model, and publishes the results to an interactive Streamlit dashboard. The solution is containerised with Docker, scheduled with cron, continuously integrated with GitHub Actions and operated on a virtual private server administered over SSH. For the 2021/22–2025/26 reference period, the model estimates a technical potential of 80.1 TWh/year (about 58 % of national electricity demand) and a gross benefit of ARS 11.2 trillion per year.

Keywords: agricultural residues; gasification; distributed generation; ETL; web scraping; data engineering; Docker; continuous integration; SSH; dashboard.

Agradecimientos

[Sección a completar por el autor.]

Índice

- Introducción
- Capítulo 1. Estado del arte y marco teórico
- Capítulo 2. Metodología: cómo se desarrolló el trabajo
- Capítulo 3. Arquitectura del sistema
- Capítulo 4. Extracción de datos agrícolas
- Capítulo 5. Extracción automática de cuadros tarifarios
- Capítulo 6. Transformación de los datos y modelo de cálculo
- Capítulo 7. Calidad del software y de los datos
- Capítulo 8. Infraestructura, despliegue y operación
- Capítulo 9. Tablero de visualización
- Capítulo 10. Resultados
- Capítulo 11. Conclusiones y trabajos futuros
- Referencias
- Anexos

Índice de figuras

N.º	Título	Capítulo
1	Arquitectura de SSH y secuencia de una conexión	1
2	Actividad del repositorio por día	2
3	Distribución de commits por tipo	2
4	Arquitectura general del sistema	3
5	Cadena de cálculo del modelo	3
6	Flujo del extractor de tarifas	5
7	Tarifas de compra e inyección por distribuidora	5
8	Página «Auditoría de datos» del tablero	7
9	Topología de despliegue en el VPS	8
10	Integración continua y evolución del despliegue	8
11	Página de inicio del tablero	9
12	Página «Resumen»	9
13	Página «Energía»	9
14	Página «Dinero»	9
15	Panel de supuestos ajustables	9
16	Página «Exploración»	9
17	Biomasa residual por campaña (serie histórica)	10

N.º	Título	Capítulo
18	Biomasa residual anual por provincia	10
19	Energía potencial por cultivo	10
20	Mapa de energía generable por departamento	10
21	Beneficio bruto anual por provincia	10
22	Mapa de beneficio bruto por provincia	10
23	Sensibilidad del beneficio a la fracción retirada	10

Lista de siglas

Sigla	Significado
API	*Application Programming Interface*, interfaz de programación de aplicaciones
BCCBA	Bolsa de Cereales de Córdoba
BT / MT	Baja tensión / media tensión
CAMMESA	Compañía Administradora del Mercado Mayorista Eléctrico
CI / CD	Integración continua / entrega (o despliegue) continua
CSV	*Comma-Separated Values*
ETL	*Extract, Transform, Load* (extracción, transformación y carga)
GADM	*Database of Global Administrative Areas*
IC	Índice de cosecha (*harvest index*)
FR	Factor residuo/producto (*residue-to-product ratio*)
MAGyP	Ministerio de Agricultura, Ganadería y Pesca de la Nación
MS	Materia seca
PP30	*Power Pallet 30*, gasificador de All Power Labs
POL-…	Identificador de una decisión metodológica registrada
SSH	*Secure Shell*, protocolo de acceso remoto seguro
TLS	*Transport Layer Security*
VPS	*Virtual Private Server*, servidor privado virtual
VPN	*Virtual Private Network*, red privada virtual

Introducción

Planteamiento del problema

La matriz eléctrica argentina depende mayoritariamente de combustibles fósiles, mientras que el sector agrícola genera todos los años un volumen enorme de residuos de cosecha que, en su mayor parte, permanece en el lote. Esa biomasa es un recurso energético renovable y distribuido territorialmente justamente donde se encuentran muchos usuarios rurales de electricidad. Sin embargo, responder preguntas aparentemente simples —*¿cuánto rastrojo hay en un departamento?, ¿cuánta electricidad podría generarse con él?, ¿cuánto dinero representaría para un productor?*— exige combinar fuentes de información públicas que están dispersas, en formatos incompatibles, con nomenclaturas distintas y que cambian con el tiempo.

Las estadísticas agrícolas se publican por departamento y campaña en portales del Estado nacional y de entidades privadas; los parámetros agronómicos que permiten convertir granos en residuo están en la bibliografía científica; el rendimiento de un equipo de conversión se encuentra en fichas técnicas de fabricantes; y el valor económico de la electricidad depende de los cuadros tarifarios de 28 distribuidoras, cada una de las cuales los publica en su propio formato (documentos PDF, planillas Excel, páginas web dinámicas o, incluso, imágenes escaneadas). Ninguna de estas fuentes está pensada para ser combinada con las demás, y cualquier estimación hecha a mano queda desactualizada en semanas.

El problema que aborda este trabajo es, por lo tanto, doble: **un problema de dominio** (estimar el potencial energético y económico de la biomasa residual agrícola con un modelo defendible) y **un problema de ingeniería de sistemas de información** (construir una infraestructura de datos que obtenga, integre, valide, calcule y publique esa estimación de forma automática, trazable y sostenible en el tiempo).

Justificación

Desde el punto de vista energético, la Ley 27.424 (2017) habilita a los usuarios a generar electricidad renovable para autoconsumo e inyectar excedentes a la red, lo que convierte al rastrojo en un activo potencialmente valorizable a escala de establecimiento. Desde el punto de vista informático, el trabajo ejercita de manera integrada las competencias centrales de la carrera: análisis de requerimientos, modelado de datos, programación, extracción e integración de datos heterogéneos, pruebas de software, administración de infraestructura, seguridad, despliegue y visualización. Disponer de un sistema que recalcule las estimaciones cada semana, documente cada decisión y exponga los resultados en un tablero interactivo reduce el costo de producir la información y aumenta su transparencia.

Objetivos

Objetivo general. Diseñar, construir y poner en operación un sistema de información que estime, para cada departamento de la Argentina, la biomasa residual agrícola, la energía eléctrica que podría generarse con ella mediante gasificación de pequeña escala y el beneficio económico asociado bajo el régimen de generación distribuida, a partir de fuentes públicas actualizadas automáticamente.

Objetivos específicos.

1. Relevar el estado del arte sobre estimación de residuos agrícolas, conversión energética por gasificación, generación distribuida e ingeniería de datos.
2. Desarrollar extractores automáticos y tolerantes a fallos para las estadísticas agrícolas del MAGyP y de la Bolsa de Cereales de Córdoba.
3. Desarrollar un extractor de cuadros tarifarios capaz de leer las publicaciones de las 28 distribuidoras eléctricas del país, con trazabilidad del documento de origen.
4. Implementar un modelo de cálculo lineal y auditable que transforme producción en biomasa, energía y dinero, con cada supuesto declarado y justificado.
5. Asegurar la calidad del software y de los datos mediante pruebas automatizadas, invariantes, alertas y notificaciones.
6. Empaquetar, programar y desplegar el sistema en contenedores sobre un servidor accesible de manera segura.
7. Construir un tablero web que permita explorar los resultados y modificar los supuestos más inciertos.

Alcance y limitaciones

El sistema calcula un **potencial técnico** y un **beneficio bruto**: no descuenta costos de inversión, operación, recolección ni transporte del rastrojo, por lo que no constituye una evaluación de factibilidad económica. Se consideran los cultivos para los que existen coeficientes agronómicos publicados (17 cultivos; 15 con producción en el período de referencia), lo que excluye, entre otros, la caña de azúcar y la yerba mate. La tecnología de conversión se fija en un único equipo comercial (APL PP30). Tres distribuidoras (Santa Cruz, Tierra del Fuego y Tucumán) no publican cuadros legibles por máquina y Chubut carece de una distribuidora provincial única, por lo que esas provincias tienen energía estimada pero no valorización económica.

Estructura del documento

El **Capítulo 1** presenta el estado del arte y el marco teórico, tanto del dominio energético como de las tecnologías informáticas empleadas, incluida una explicación de SSH desde sus

fundamentos. El **Capítulo 2** describe la metodología: cómo se organizó el trabajo, en qué fases, con qué herramientas y con qué criterios de decisión. Los **Capítulos 3 a 9** detallan cada parte del sistema: arquitectura, extracción agrícola, extracción tarifaria, transformación y modelo de cálculo, calidad, infraestructura y tablero. El **Capítulo 10** expone los resultados y el **Capítulo 11** reúne las conclusiones generales y las mejoras a futuro. Al final se listan las referencias y se incluyen anexos con el registro de decisiones, la notación y los comandos de operación. Cada capítulo cierra con sus propias conclusiones parciales.

Capítulo 1. Estado del arte y marco teórico

Este capítulo reúne los conceptos y antecedentes sobre los que se apoya el trabajo. Se organiza en dos grandes bloques: el **dominio** (biomasa, conversión energética y regulación eléctrica) y las **tecnologías de sistemas de información** utilizadas para resolver el problema (ingeniería de datos, contenedores, integración continua, redes y seguridad, y visualización).

1.1 Biomasa residual agrícola y su cuantificación

Se denomina **biomasa residual agrícola** a la fracción aérea de la planta que no constituye el producto cosechado: tallos, hojas, chalas y marlos que quedan en el lote tras la cosecha. La forma estándar de estimarla a partir de estadísticas de producción es mediante el **índice de cosecha** (IC, *harvest index*), definido como la proporción del grano en la biomasa aérea total (Hay, 1995; Unkovich et al., 2010). Si P es la producción de grano, la biomasa aérea total es P/IC y el residuo es $P{\cdot}(1–IC)/IC$. El cociente $FR = (1–IC)/IC$ se conoce como **factor residuo/producto** (*residue-to-product ratio*, RPR) y es la herramienta habitual en los inventarios de residuos a gran escala (Scarlat et al., 2010; Lal, 2005).

Dos precisiones metodológicas son críticas. La primera es la **base de humedad**: la producción se informa con la humedad comercial del grano, por lo que, para obtener masa energéticamente útil, debe expresarse en **materia seca** (MS) multiplicando por la fracción seca del producto. La segunda es la **fracción recuperable**: no todo el rastrojo puede retirarse, porque cumple funciones agronómicas esenciales (protección contra la erosión, aporte de carbono orgánico, conservación de la humedad). La literatura sitúa la fracción retirable sostenible en un rango amplio, típicamente entre 30 % y 50 % según suelo, clima y labranza (Wilhelm et al., 2004, 2007; Blanco-Canqui y Lal, 2009).

Los estudios de recursos bioenergéticos distinguen, además, niveles anidados de **potencial**: teórico (toda la biomasa producida), técnico (lo que puede retirarse y convertirse con una tecnología dada respetando restricciones ambientales), económico (la parte rentable) y de mercado (lo que efectivamente se adoptaría) (Batidzirai et al., 2012; Lopez et al., 2012). Batidzirai et al. (2012) señalan que la mayor fuente de dispersión entre estudios es la falta de explicitación de supuestos, observación que guió la decisión de registrar cada supuesto de este trabajo con un identificador propio.

Antecedentes en la Argentina. El estudio WISDOM Argentina (FAO, 2009), actualizado en 2020, construyó un balance espacial de oferta y demanda de biomasa con fines energéticos y dio origen al programa PROBIOMASA. A diferencia de WISDOM —orientado a dendroenergía y a cortes temporales puntuales—, OPTIMIA-HV trabaja con datos de cosecha por departamento y campaña que se actualizan automáticamente, lleva el cálculo hasta energía

eléctrica con una tecnología concreta y valoriza esa energía con las tarifas vigentes de cada distribuidora.

1.2 Conversión energética por gasificación

Las tecnologías de conversión de biomasa en energía se agrupan en termoquímicas (combustión, gasificación, pirólisis) y bioquímicas (digestión anaeróbica, fermentación) (McKendry, 2002a). La **gasificación** es la oxidación parcial de la biomasa a alta temperatura, que produce un gas combustible (*syngas*, compuesto principalmente por CO, H_2 y CH_4) apto para alimentar un motor de combustión interna acoplado a un generador (McKendry, 2002b; Basu, 2018). Los gasificadores de **lecho descendente** (*downdraft*) se prefieren a pequeña escala porque producen un gas con bajo contenido de alquitranes, compatible con motores (Reed y Das, 1988).

El equipo de referencia adoptado es el **Power Pallet PP30** de All Power Labs, un sistema comercial integrado (gasificador *downdraft*, motor y generador) cuya ficha técnica declara una potencia de 22 kW a 50 Hz y un consumo específico de aproximadamente 1,0 kg de biomasa seca por kWh (All Power Labs, s.f.-a). El fabricante aprueba como combustible chips de madera, cáscaras y marlos; el rastrojo suelto requiere acondicionamiento (triturado o pelletizado), cuya necesidad se discute en la literatura sobre densificación (Tumuluru et al., 2011). Las propiedades del combustible (humedad, cenizas, composición) condicionan el desempeño (Jenkins et al., 1998; Vassilev et al., 2010).

1.3 Generación distribuida y valorización de la energía

La **Ley 27.424** establece el Régimen de Fomento a la Generación Distribuida de Energía Renovable Integrada a la Red Eléctrica Pública, reglamentado por el Decreto 986/2018 y la Resolución 314/2018. Bajo este régimen, un usuario-generador consume su propia energía (lo que le ahorra la tarifa de compra) e inyecta el excedente, que la distribuidora le reconoce a una **tarifa de inyección**. El esquema es un **balance neto de facturación** (*net billing*), en el que la energía inyectada se valoriza a un precio distinto —en general menor— que la consumida, a diferencia del *net metering* puro (Darghouth et al., 2011). Por esa razón, el valor económico de un kWh generado depende de qué fracción se autoconsume y de qué fracción se inyecta, y de las dos tarifas de la distribuidora local.

Las tarifas eléctricas argentinas se publican en **cuadros tarifarios** emitidos por cada distribuidora o ente regulador provincial, con categorías por tipo de usuario, nivel de tensión y franja horaria. No existe un formato común ni un servicio de datos unificado: la Secretaría de Energía de la Nación republica parte de ellos, pero con demoras y omisiones. Este hecho convierte la obtención sistemática de tarifas en uno de los desafíos técnicos centrales del trabajo.

1.4 Ingeniería de datos

Procesos ETL. La integración de datos provenientes de sistemas heterogéneos se resuelve tradicionalmente con procesos de extracción, transformación y carga (Kimball y Caserta, 2004). La extracción obtiene los datos de sus fuentes; la transformación los limpia, normaliza, integra y enriquece; y la carga los deposita en un almacén apto para el análisis. Las arquitecturas contemporáneas organizan los datos en capas de madurez creciente —denominadas a menudo **bronce, plata y oro**—, en las que la capa bronce conserva los datos crudos tal como se obtuvieron, lo que permite reprocesar sin volver a descargar y auditar cualquier resultado (Armbrust et al., 2021).

Extracción web (*web scraping*). Cuando los datos no se ofrecen mediante una API, se obtienen automatizando la descarga y el análisis de páginas y documentos publicados (Mitchell, 2018). Esta práctica exige respetar la carga de los servidores, manejar fallos transitorios y detectar cambios de formato. La técnica estándar ante fallos transitorios es el **reintento con espera exponencial** (*exponential backoff*), que duplica el tiempo de espera entre intentos para no agravar la congestión (Brooker, 2015), respetando las indicaciones del servidor como el encabezado `Retry-After` y el código de estado 429 (*Too Many Requests*) (Nottingham y Fielding, 2012; Fielding et al., 2022).

Integración de entidades. Combinar registros de distintas fuentes requiere decidir cuándo dos nombres refieren a la misma entidad (por ejemplo, «Gral. San Martín» y «General San Martín»). Este problema se conoce como *record linkage* o emparejamiento de datos (Christen, 2012) y se resuelve con normalización léxica, diccionarios de alias y reglas explícitas de desempate.

Almacenamiento columnar. Apache Parquet es un formato de archivo columnar, comprimido y autodescriptivo, derivado de las ideas del sistema Dremel (Melnik et al., 2010), que resulta muy eficiente para cargas analíticas porque solo lee las columnas necesarias. **DuckDB** es un motor SQL analítico embebido, que se ejecuta dentro del proceso de la aplicación sin servidor y consulta archivos Parquet directamente (Raasveldt y Mühleisen, 2019). La combinación permite prescindir de un servidor de base de datos cuando el volumen es moderado. Para la manipulación de datos en Python se utiliza la biblioteca **pandas** (McKinney, 2010).

1.5 Contenedores y orquestación

Un **contenedor** es un proceso aislado que empaqueta una aplicación junto con todas sus dependencias, de modo que se ejecuta de la misma forma en cualquier máquina con un motor compatible. **Docker** popularizó este modelo mediante imágenes inmutables construidas por capas a partir de un `Dockerfile` (Merkel, 2014). **Docker Compose** permite declarar en un

único archivo varios servicios, sus volúmenes y sus redes. La metodología *Twelve-Factor App* recomienda separar estrictamente código, configuración (por variables de entorno) y datos, y tratar los procesos como efímeros (Wiggins, 2011); varios de sus principios guiaron el diseño del despliegue. La ejecución periódica de tareas se resuelve con la notación **cron**, que expresa calendarios mediante campos de minuto, hora, día del mes, mes y día de la semana.

1.6 Control de versiones, integración y entrega continuas

Git es un sistema de control de versiones distribuido que registra la historia completa de un proyecto como una secuencia de *commits* y permite trabajar en ramas paralelas que luego se integran (Chacon y Straub, 2014). La convención **Conventional Commits** estructura los mensajes con un tipo (`feat`, `fix`, `docs`, etc.) que hace legible la historia. La **integración continua** (CI) consiste en integrar los cambios con frecuencia y verificar cada integración con una compilación y pruebas automáticas (Fowler, 2006); la **entrega continua** (CD) extiende esa automatización hasta el despliegue (Humble y Farley, 2010). Las pruebas automatizadas constituyen la red de seguridad que vuelve posible el cambio frecuente (Myers et al., 2011). Por último, los **registros de decisiones de arquitectura** documentan cada decisión relevante con su contexto, alternativas y consecuencias (Nygard, 2011), práctica que este trabajo adopta bajo la forma de reglas `POL-...`.

1.7 Redes y seguridad: SSH desde sus fundamentos

Operar un sistema en un servidor remoto exige un canal seguro para administrarlo. El estándar para ello es **SSH** (*Secure Shell*). Dado que es una pieza central de la operación de este trabajo, se lo explica a continuación desde sus fundamentos.

1.7.1 El problema que resuelve SSH

Antes de SSH, la administración remota se hacía con protocolos como Telnet o `rlogin`, que transmitían todo —incluidas las contraseñas— en texto plano: cualquier equipo intermedio en la red podía leerlo o modificarlo. En 1995, tras un ataque de captura de contraseñas en la red de la Universidad Tecnológica de Helsinki, Tatu Ylönen diseñó SSH como un reemplazo que garantizara tres propiedades sobre una red insegura (Ylonen, 1996):

- **Confidencialidad**: nadie más que los extremos puede leer el contenido.
- **Integridad**: cualquier alteración del contenido en tránsito se detecta.
- **Autenticación**: el cliente puede verificar que habla con el servidor correcto, y el servidor puede verificar la identidad del usuario.

1.7.2 Fundamentos criptográficos

SSH combina tres familias de técnicas criptográficas (Stallings, 2017; Menezes et al., 1996):

1. **Criptografía simétrica.** Ambos extremos comparten una misma clave secreta con la que cifran y descifran (por ejemplo, AES o ChaCha20). Es muy rápida, pero plantea un problema: ¿cómo acuerdan la clave dos partes que nunca se comunicaron, si el canal es inseguro?
2. **Intercambio de claves de Diffie-Hellman.** Diffie y Hellman (1976) mostraron que dos partes pueden derivar un secreto compartido intercambiando únicamente valores públicos, de modo que un observador que ve todo el intercambio no puede calcular el secreto. La variante moderna usa curvas elípticas (ECDH), en particular **Curve25519** (Bernstein, 2006), adoptada en SSH por el RFC 8731 (Adamantiadis et al., 2020).
3. **Criptografía asimétrica y firmas digitales.** Cada parte posee un par de claves: una **privada**, que nunca se comparte, y una **pública**, que puede distribuirse libremente. Lo que se firma con la clave privada puede verificarse con la pública, pero no puede falsificarse sin la privada (Rivest et al., 1978). SSH usa firmas para autenticar al servidor y, opcionalmente, al usuario. El algoritmo moderno recomendado es **Ed25519** (Bernstein et al., 2012), incorporado a SSH por el RFC 8709.

1.7.3 Arquitectura del protocolo

La versión estandarizada de SSH (SSH-2) se define en una familia de documentos del IETF y se organiza en tres capas apiladas sobre TCP (Ylonen y Lonvick, 2006a):

- **Capa de transporte** (RFC 4253): establece la conexión cifrada. Negocia algoritmos, ejecuta el intercambio de claves, autentica al **servidor** mediante su *host key* y, a partir de allí, cifra y protege la integridad de todo el tráfico (Ylonen y Lonvick, 2006c).
- **Capa de autenticación de usuario** (RFC 4252): una vez establecido el canal cifrado, el cliente demuestra su identidad. Los métodos más comunes son contraseña y **clave pública** (Ylonen y Lonvick, 2006b).
- **Capa de conexión** (RFC 4254): multiplexa sobre el túnel cifrado varios **canales** lógicos independientes: una terminal interactiva, la ejecución de un comando puntual, el reenvío de puertos o la transferencia de archivos (Ylonen y Lonvick, 2006d).

1.7.4 Secuencia de una conexión

La Figura 1 resume la secuencia de una conexión típica. (1) El cliente abre una conexión TCP al puerto 22 del servidor y ambos intercambian sus versiones. (2) Cada uno envía la lista de algoritmos que soporta y se elige el primero en común. (3) Se ejecuta el intercambio de claves ECDH, del que ambos extremos derivan el mismo secreto sin haberlo transmitido. (4) El servidor firma los datos del intercambio con su clave privada de *host*; el cliente verifica la firma con la clave pública del servidor, que compara con la registrada en su archivo

`known_hosts`. Si la clave no coincide, el cliente advierte un posible ataque de intermediario (*man-in-the-middle*). (5) A partir del mensaje `NEWKEYS`, todo el tráfico viaja cifrado con claves simétricas derivadas del secreto. (6) El usuario se autentica: con el método de clave pública, el cliente firma un desafío con la clave privada del usuario. (7) El servidor verifica la firma contra las claves públicas autorizadas en el archivo `authorized_keys` de esa cuenta. (8) Se abre un canal de sesión, en el que se ejecuta un comando o una terminal interactiva.

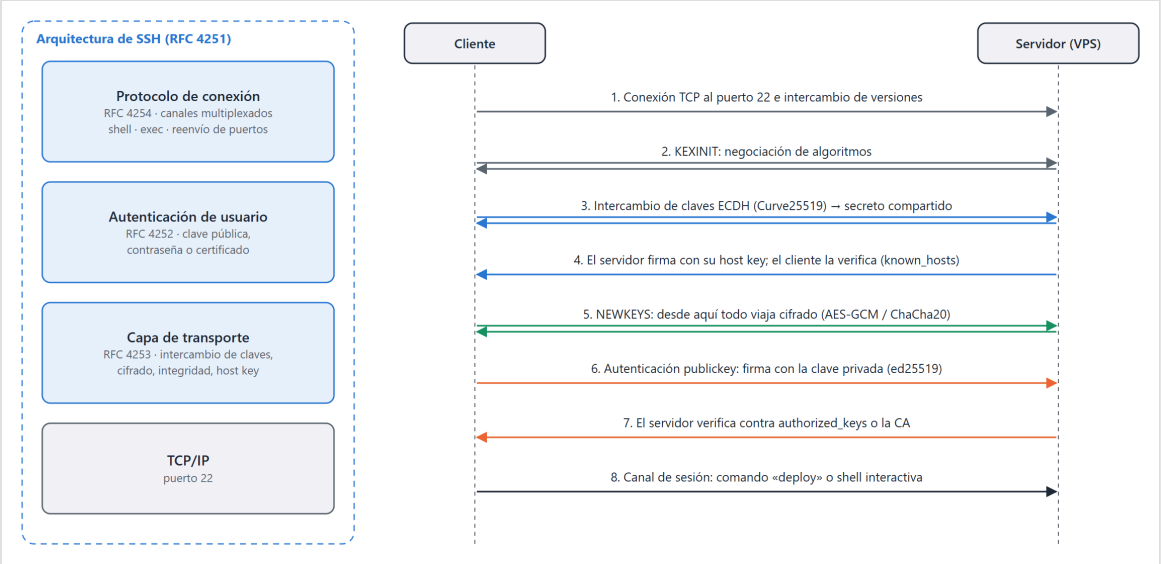


Figura 1. Arquitectura en capas de SSH y secuencia de una conexión con autenticación por clave pública.
Fuente: elaboración propia a partir de Ylonen y Lonvick (2006a–d).

1.7.5 Prácticas de endurecimiento

La autenticación por clave pública es preferible a la contraseña porque la clave privada nunca viaja por la red y no puede adivinarse por fuerza bruta (Barrett et al., 2005). Sobre esta base se aplican prácticas complementarias que este trabajo utiliza (Capítulo 8):

- **Comandos forzados**: en `authorized_keys` puede asociarse a una clave la opción `command="…"` junto con `restrict`, de modo que esa clave solo pueda ejecutar un comando determinado, sin terminal ni reenvío de puertos. Es la forma de dar a un sistema automatizado (por ejemplo, un *pipeline* de despliegue) un acceso mínimo.
- **Certificados SSH**: en lugar de registrar cada clave pública, el servidor confía en una autoridad certificante (`TrustedUserCAKeys`) que firma claves de usuario con vigencia acotada, lo que permite otorgar accesos que expiran solos (The OpenBSD Project, s.f.).
- **Cortafuegos y VPN**: restringir los puertos expuestos y acceder a SSH a través de una red privada virtual reduce la superficie de ataque. **WireGuard** es un protocolo de VPN moderno, simple y verificable (Donenfeld, 2017), sobre el que se construyen servicios de red privada como Tailscale.

1.7.6 HTTPS y proxy inverso

El acceso de los usuarios finales a una aplicación web se protege con **TLS** (Rescorla, 2018), que cumple para HTTP una función análoga a la que la capa de transporte cumple en SSH. Los certificados se obtienen automáticamente de autoridades como **Let's Encrypt** mediante el protocolo ACME (Aas et al., 2019). Un **proxy inverso** recibe las conexiones HTTPS públicas, termina TLS y las reenvía a la aplicación interna, que así no necesita exponerse directamente.

1.8 Visualización de datos y tableros

Un **tablero** (*dashboard*) presenta en una sola vista la información necesaria para comprender una situación de un vistazo (Few, 2006). Su eficacia depende de principios de diseño de gráficos cuantitativos: maximizar la proporción de tinta dedicada a los datos y evitar decoraciones que distorsionen la lectura (Tufte, 2001). Para la información territorial se utilizan **mapas coropléticos**, en los que cada unidad administrativa se colorea según el valor de una variable; la elección de escalas de color secuenciales de un solo tono facilita la lectura y la accesibilidad (Harrower y Brewer, 2003). Los límites administrativos se obtienen de la base GADM (GADM, 2022). Herramientas como **Streamlit** permiten construir tableros interactivos íntegramente en Python, y **Plotly** provee gráficos interactivos en el navegador.

1.9 Conclusiones del capítulo

El relevamiento muestra que los fundamentos del dominio están consolidados —el índice de cosecha para estimar residuos, la fracción retirable para respetar el suelo, la gasificación *downdraft* para la conversión a pequeña escala y el balance neto para la valorización—, pero que su aplicación sistemática en la Argentina choca con un obstáculo informático: la dispersión y heterogeneidad de las fuentes. Del lado tecnológico, existen herramientas maduras para cada eslabón (ETL por capas, extracción web resiliente, almacenamiento columnar, contenedores, integración continua, acceso remoto seguro mediante SSH y tableros interactivos). La contribución de este trabajo no reside en inventar ninguna de esas piezas, sino en **integrarlas** en un sistema coherente, automatizado y auditable que resuelva el problema de punta a punta. El capítulo siguiente describe cómo se organizó ese trabajo de integración.

Capítulo 2. Metodología: cómo se desarrolló el trabajo

2.1 Tipo de investigación y enfoque

El trabajo corresponde a una **investigación aplicada de desarrollo tecnológico** con enfoque **cuantitativo** (Hernández Sampieri et al., 2014): se construye un artefacto —un sistema de información— que produce mediciones numéricas a partir de datos secundarios públicos y de parámetros tomados de la bibliografía. La validez de los resultados se apoya en tres pilares: la trazabilidad de cada dato hasta su fuente, la justificación bibliográfica de cada supuesto y la verificación automática de las propiedades que el cálculo debe cumplir.

Como proceso de ingeniería de software, se adoptó un **ciclo de vida iterativo e incremental** (Larman y Basili, 2003; Sommerville, 2011): el sistema creció en incrementos funcionales —primero la biomasa, luego la energía, luego el dinero, luego la automatización y el despliegue—, y cada incremento se validó contra fuentes reales antes de construir el siguiente. Este enfoque resultó adecuado porque los requerimientos se descubrían a medida que se exploraban las fuentes: la forma en que cada distribuidora publica su cuadro tarifario, por ejemplo, solo se conoció al intentar leerlo.

2.2 Fases del desarrollo

El historial del repositorio —203 *commits* entre el 10 de septiembre y el 11 de noviembre de 2026, concentrados en 17 días de trabajo efectivo— permite reconstruir con precisión las fases del proyecto (Figura 2 y Tabla 1).

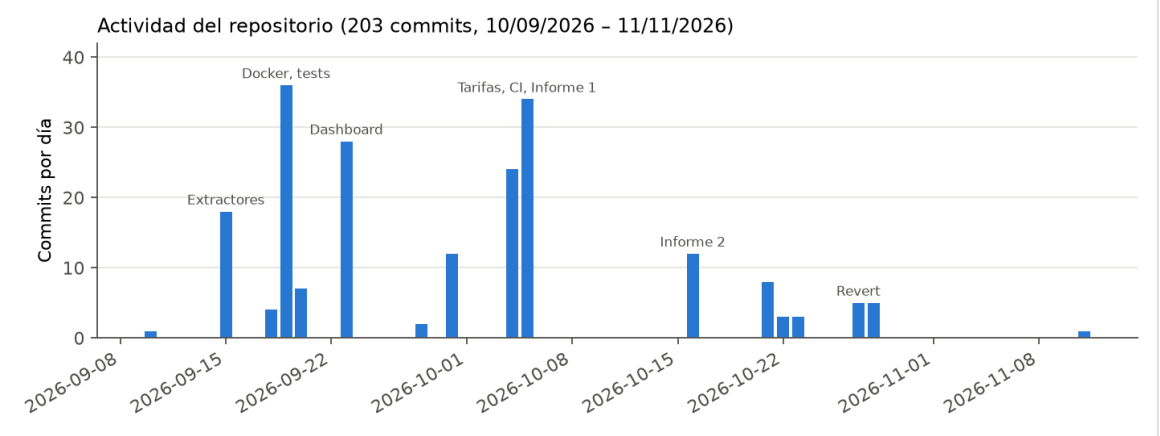


Figura 2. Actividad del repositorio: commits por día. Fuente: elaboración propia a partir de `git log`.

Tabla 1. Fases del desarrollo reconstruidas a partir del historial de commits.

Fase	Fechas	Contenido principal	Commits representativos
1. Andamiaje y exploración	10–19/09	Estructura del repositorio, extractores de MAGyP y bolsas de cereales, datos crudos, *notebook* exploratorio, Docker, configuración, primeras pruebas, diagramas y registro de decisiones	`bd58567`, `668608a`, `97316ca`, `dcf7615`
2. Transformación y tablero inicial	20–23/09	Transformadores (biomasa, cultivos, geografía, reconciliación, validación), servicio programado con Ofelia, scripts de operación, primer tablero Streamlit, polígonos GADM	`b292d5b`, `d872385`, `1201230`
3. Energía, dinero y tarifas	30/09	Primer extractor de tarifas, transformación energética y financiera, etapa de extracción en el orquestador	`1d271fb`, `5a15866`, `e9218b3`
4. Validación contra fuentes reales	4–5/10	Corrección de errores silenciosos, 28 distribuidoras con historial, parser por puntaje, tablero de energía y dinero, reintentos, integración continua, reorganización de la documentación, Informe de avance 1	`01dc552`, `12ac70e`, `9b4cb7f`, `5e45227`
5. Refinamiento del modelo	5–16/10	Avisos por Telegram/webhook, nuevas distribuidoras (EPEC, ESJ, EJESA, EDERSA, EDESA, APE), ficha técnica del PP30, supuestos ajustables, Informe de avance 2	`98ac168`, `63e96ad`, `e8f87a2`, `57186c5`
6. Automatización del flujo y despliegue	16–28/10	Fusión automática de PR, reestructuración en monorepo, despliegue continuo (SSH, Dokploy) y su reversión	`b2f94e2`, `874fefa`, `977ab5f`, `63cbb88`
7. Documentación de cierre	27–28/10	Inventario de unidades, informe de tarifas faltantes, fe de erratas	`84b1ffe`, `e340750`, `946ed5d`
8. Ajuste operativo	11/11	Deduplicación de alertas y ejecución semanal	`aae81eb`

Fuente: elaboración propia.

La Tabla 1 muestra un patrón característico del desarrollo guiado por datos: después de construir la primera versión completa de la cadena (fases 1 a 3), una fase entera (la 4) se dedicó a **confrontar los resultados con las fuentes originales**, lo que reveló errores que ninguna prueba unitaria habría detectado por sí sola (Sección 7.5).

2.3 Gestión de la configuración

El código se versionó con Git y se alojó en GitHub. Hasta fines de septiembre los cambios se integraron directamente en la rama principal; a partir de octubre, todo cambio llegó a `main` mediante una **rama temática y una solicitud de incorporación** (*pull request*), verificada por la integración continua y, desde el 16 de octubre, fusionada automáticamente cuando las pruebas pasaban (Sección 8.6). Se registraron 25 fusiones de *pull requests*. Los mensajes de

commit siguen la convención Conventional Commits, cuya distribución se presenta en la Figura 3: predominan las nuevas funcionalidades (`feat`, 42 % de los commits que no son fusiones), seguidas por refactorizaciones, tareas de mantenimiento, correcciones y documentación.

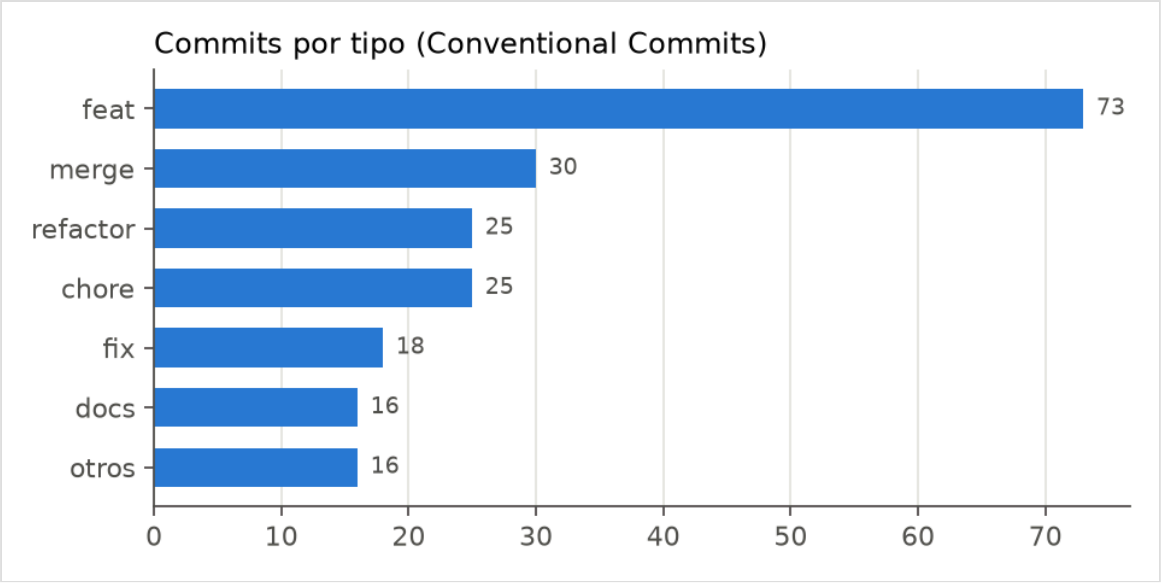


Figura 3. Distribución de commits por tipo. Fuente: elaboración propia a partir de `git log`.

2.4 Registro de decisiones metodológicas

Inspirado en los registros de decisiones de arquitectura (Nygard, 2011), cada decisión que afecta los resultados se documentó con un identificador `POL-…` en `docs/02_metodologia/decisiones.md`, con su contexto, la regla adoptada, la justificación y las consecuencias. El identificador se cita en el código fuente en el lugar donde la regla se implementa, lo que vuelve auditable la correspondencia entre documento y programa. Se registraron diez decisiones, resumidas en el Anexo A; por ejemplo, `POL-SOURCE-001` define cómo elegir entre dos fuentes que informan lo mismo y `POL-TARIFF-002` cómo estimar una tarifa de inyección faltante.

2.5 Herramientas utilizadas

Tabla 2. Herramientas y tecnologías empleadas.

Ámbito	Herramienta	Uso en el trabajo
Lenguaje	Python 3.11	Todo el sistema (ETL, tablero, pruebas)
Datos	pandas, NumPy, PyArrow	Transformación y escritura de Parquet
Consulta	DuckDB	Lectura de Parquet desde el tablero
Extracción	httpx, Beautiful Soup, curl_cffi	Descarga y análisis de páginas

Ámbito	Herramienta	Uso en el trabajo
Documentos	pypdf, openpyxl, xlrd	Lectura de PDF, XLSX y XLS
Visualización	Streamlit, Plotly	Tablero web y mapas coropléticos
Exploración	Jupyter	*Notebook* de análisis y verificación
Calidad	pytest, ruff	Pruebas automatizadas y análisis estático
Contenedores	Docker, Docker Compose	Empaquetado y orquestación
Planificación	Ofelia (cron)	Ejecución semanal del ETL
CI	GitHub Actions	Pruebas, construcción de imagen y fusión automática
Servidor	VPS Debian, Caddy, ufw, Tailscale, OpenSSH	Operación segura en producción
Asistencia	Claude Code (Anthropic)	Asistente de programación supervisado

Fuente: elaboración propia.

2.6 Desarrollo asistido por inteligencia artificial

Por honestidad académica corresponde declarar que, a partir del 4 de octubre de 2026, parte del desarrollo se realizó con la asistencia de un agente de programación basado en un modelo de lenguaje (Claude Code, de Anthropic). El historial de Git lo registra de forma explícita: 52 commits figuran con ese agente como autor y otros llevan la marca de coautoría correspondiente. La modalidad de trabajo fue **supervisada**: el autor definió los objetivos de cada incremento, revisó y validó los cambios propuestos contra las fuentes reales, decidió los supuestos metodológicos y aprobó cada integración mediante *pull requests*. El agente aceleró tareas como la escritura de lectores específicos para cada formato de cuadro tarifario, la ampliación de las pruebas y la redacción de documentación técnica. La responsabilidad sobre el diseño, las decisiones y los resultados es del autor.

2.7 Criterios de validación

Se adoptaron cuatro criterios para dar por válido un incremento:

1. **Contraste con la fuente**: todo valor extraído (una producción, una tarifa) se comparó, para una muestra, con el documento original.
2. **Pruebas automatizadas**: cada corrección de error se acompañó de una prueba de regresión que lo reproduce.
3. **Invariantes de datos**: propiedades que deben cumplirse siempre (por ejemplo, que no haya registros duplicados tras reconciliar) se verifican en cada ejecución del ETL.
4. **Ejemplo trazable**: el cálculo completo se reprodujo a mano para un caso de referencia (10.000 t de maíz) y se comparó con el resultado del sistema (Sección 6.8).

2.8 Conclusiones del capítulo

El trabajo se condujo como un desarrollo iterativo e incremental en ocho fases, cuya secuencia está documentada en el historial de versiones. Tres decisiones metodológicas resultaron determinantes: registrar cada supuesto con un identificador citado en el código, validar contra las fuentes reales antes de avanzar y formalizar el flujo de integración mediante ramas, *pull requests* y pruebas automáticas. Estas prácticas convierten el sistema en un objeto auditable: cualquier cifra puede rastrearse hasta el documento del que proviene y hasta la regla que la produjo.

Capítulo 3. Arquitectura del sistema

3.1 Requerimientos

Del planteamiento del problema se derivaron los siguientes requerimientos.

Funcionales. (RF1) Obtener la producción agrícola por departamento, cultivo y campaña. (RF2) Obtener las tarifas de compra e inyección de cada distribuidora. (RF3) Calcular biomasa, energía, equipos equivalentes y beneficio. (RF4) Expresar los resultados como promedio anual de un período de referencia. (RF5) Mostrar los resultados en mapas, rankings y tablas, con filtros. (RF6) Permitir modificar los supuestos más inciertos sin recalcular el ETL. (RF7) Exponer los datos descartados y las alertas para su auditoría.

No funcionales. (RNF1) Actualización automática y periódica. (RNF2) Tolerancia a la caída de cualquier fuente individual. (RNF3) Trazabilidad de cada valor hasta su documento de origen. (RNF4) Reproducibilidad del entorno de ejecución. (RNF5) Bajo costo de operación: un único servidor modesto. (RNF6) Acceso administrativo seguro. (RNF7) Mantenibilidad: código probado y documentado.

3.2 Vista general

La Figura 4 presenta la arquitectura. El sistema adopta un estilo de **tubería de datos por lotes** (*batch pipeline*) con tres componentes desacoplados que se comunican exclusivamente a través de archivos:

1. **Fuentes públicas**: el portal de estimaciones del MAGyP, la planilla de la Bolsa de Cereales de Córdoba, los sitios de las 28 distribuidoras (y de sus entes reguladores) y la base de límites administrativos GADM 4.1.
2. **ETL**: un programa Python que ejecuta ocho etapas secuenciales —extracción, limpieza y normalización, reconciliación, biomasa, energía y dinero, anualización, controles de calidad y persistencia— dentro de un contenedor Docker, disparado semanalmente por un planificador.
3. **Almacenamiento y presentación**: los resultados se escriben en archivos organizados por madurez (`raw`, `reference`, `processed`, `audit`) y un tablero Streamlit los lee mediante DuckDB.

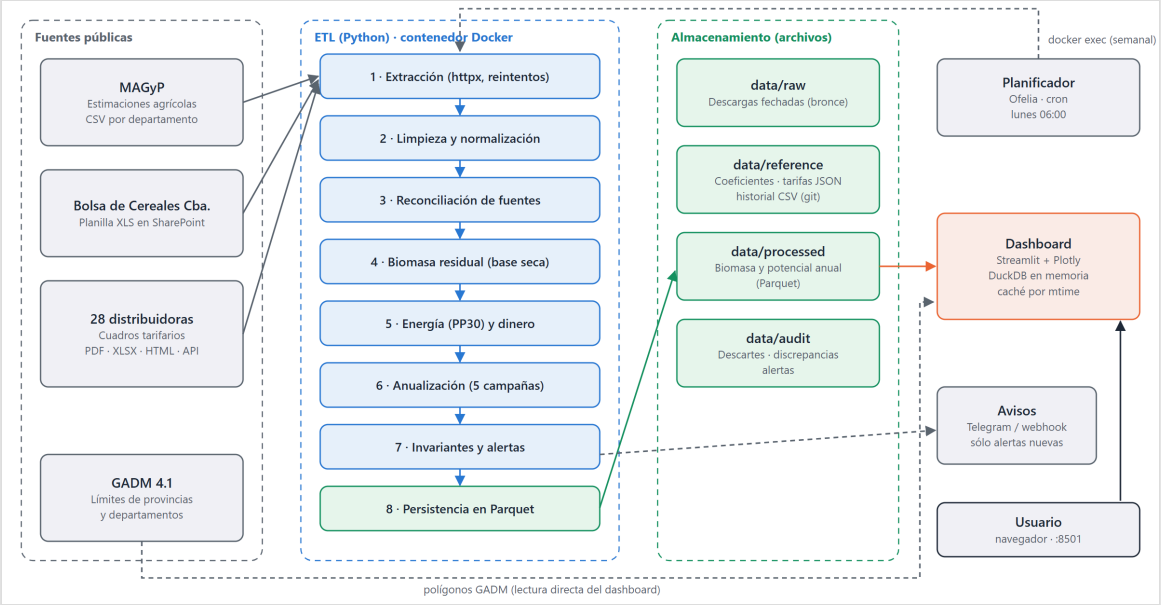


Figura 4. Arquitectura general de OPTIMIA-HV. Fuente: elaboración propia.

3.3 Decisiones de arquitectura

Archivos en lugar de un servidor de base de datos. El volumen de datos (≈110.000 registros históricos y ≈2.400 registros anuales) no justifica un servidor de base de datos. Los archivos Parquet, consultados con DuckDB embebido, ofrecen rendimiento analítico suficiente, eliminan un servicio que administrar y simplifican las copias de seguridad (RNF5).

Comunicación únicamente por archivos. El ETL y el tablero no se invocan entre sí: el ETL escribe y el tablero lee. El tablero detecta datos nuevos comparando la fecha de modificación (*mtime*) del archivo, sin necesidad de reiniciarse. Este desacople permite que un fallo del ETL no afecte la disponibilidad del tablero, que sigue mostrando la última versión válida (RNF2).

Capa cruda inmutable. Toda descarga se guarda con un prefijo de fecha y hora (`AAAAMMDDhhmmss_`) y nunca se sobrescribe. Ello permite reprocesar sin volver a descargar (opción `--skip-extract`), auditar cualquier resultado pasado y usar la última descarga válida como respaldo cuando una fuente está caída (RNF2, RNF3).

Datos de referencia versionados. Los coeficientes agronómicos, los JSON de tarifas y el historial tarifario se versionan en Git como texto legible, de modo que cada cambio de tarifa queda registrado con un *diff* revisable.

Una única imagen para todos los servicios. El ETL, el trabajador programado y el tablero comparten la misma imagen Docker, lo que garantiza que ejecutan exactamente el mismo código y las mismas dependencias (RNF4).

3.4 Organización del código

Tabla 3. Estructura del repositorio y tamaño de cada componente.

Directorio	Contenido	Archivos .py	Líneas
`etl/extractors/`	Descargas de MAGyP, BCCBA y tarifas; lector de documentos; parser de cuadros; reintentos	8	≈3.880
`etl/reference/`	Taxonomía de cultivos, carga de coeficientes, registro de distribuidoras	3	807
`etl/transformers/`	Limpieza, texto, geografía, cultivos, reconciliación, biomasa, energía y finanzas, período de referencia, validación, controles	10	≈1.950
`etl/loaders/`	Persistencia en Parquet e historial de tarifas	2	265
`etl/pipelines/`	Orquestador `run_etl.py`	1	518
`dashboard/`	Aplicación Streamlit, carga de datos, gráficos, seis vistas	10	1.692
`tests/`	Pruebas automatizadas	13	2.185
Total Python		**49**	**≈11.450**

Fuente: elaboración propia con `git ls-files` y `wc -l`. La documentación suma además 25 archivos Markdown (≈4.800 líneas).

3.5 Cadena de cálculo

El núcleo del dominio es una cadena de transformaciones **lineales** aplicada a cada registro (departamento × cultivo × campaña), que se resume en la Figura 5 y se detalla en el Capítulo 6. La linealidad es una decisión de diseño deliberada: hace transparente la contribución de cada supuesto y permite que el tablero recalcule escenarios multiplicando columnas, sin volver a ejecutar el ETL (RF6).

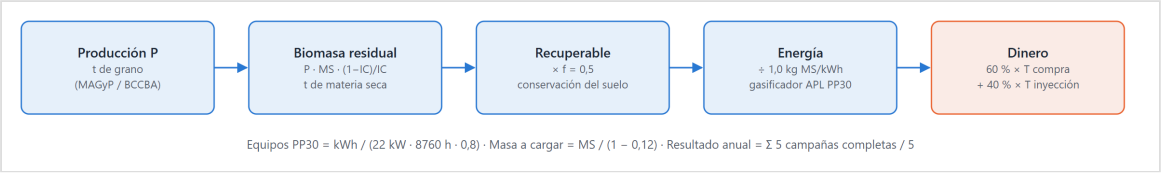


Figura 5. Cadena de cálculo: de la producción de grano al beneficio económico. Fuente: elaboración propia.

3.6 Conclusiones del capítulo

La arquitectura privilegia la simplicidad operativa y la trazabilidad por sobre la sofisticación: archivos en lugar de bases de datos, comunicación asincrónica por archivos, capa cruda inmutable y una única imagen de contenedor. Cada decisión responde a un requerimiento no funcional explícito. El resultado es un sistema que puede ejecutarse completo en una

computadora personal o en un servidor modesto, que tolera la caída de sus fuentes y cuya salida puede auditarse de punta a punta.

Capítulo 4. Extracción de datos agrícolas

4.1 Fuentes evaluadas

Tabla 4. Fuentes de datos agrícolas evaluadas.

Fuente	Institución	Formato	Granularidad	Decisión
Estimaciones Agrícolas	MAGyP	Formulario web → CSV/XLS	Departamento × cultivo × campaña, desde 1969/70, 23 provincias	Adoptada (fuente principal)
Estadística de cultivos	Bolsa de Cereales de Córdoba	Planilla XLS en SharePoint	Departamento × cultivo × campaña (Córdoba y zonas vecinas)	Adoptada (complementaria)
Panorama Agrícola Semanal	Bolsa de Cereales de Buenos Aires	XLSX tras desafío anti-bot	14 zonas agroecológicas	Descartada

Fuente: elaboración propia.

La fuente de la Bolsa de Cereales de Buenos Aires se descartó porque informa por **zonas agroecológicas** que atraviesan límites provinciales y no pueden asignarse a departamentos, y porque el sitio aplica un desafío anti-bot que obligaba a imitar la huella TLS de un navegador (`curl_cffi` con `impersonate="chrome110"`). El extractor se conserva en el repositorio, pero el orquestador no lo invoca.

4.2 Extractor del MAGyP

El portal de Estimaciones Agrícolas no ofrece una API documentada, sino un formulario web. El extractor (`etl/extractors/scrape_estimaciones_agricolas.py`, 966 líneas) automatiza el uso de ese formulario:

1. Descarga el formulario y, con Beautiful Soup, obtiene la última campaña disponible y los identificadores de todos los cultivos, en lugar de codificarlos a mano.
2. Obtiene los catálogos de provincias y departamentos de los *endpoints* AJAX del propio portal y los guarda en caché en disco.
3. Para cada departamento envía una solicitud POST con los cultivos, las variables (superficie sembrada y cosechada, producción y rendimiento) y el rango de campañas, con una pausa de 1,5 s entre solicitudes para no sobrecargar el servidor.
4. Detecta el formato de la respuesta por su firma binaria (`PK` para XLSX, `D0CF11E0` para XLS) y lee el CSV con separador `;`, coma decimal y codificación Latin-1.

5. Valida que la respuesta tenga las columnas esperadas, porque el portal puede responder con el código HTTP 200 una página HTML de error que, leída como CSV, produciría una tabla de una sola columna sin generar ningún error.

Como control de calidad, si falla más del **5 %** de los departamentos se aborta la extracción en lugar de guardar un conjunto parcial que sesgaría los resultados sin advertencia.

4.3 Extractor de la Bolsa de Cereales de Córdoba

La planilla se descarga desde un enlace público de SharePoint. El extractor (`scrape_bolsa_cereales_cba.py`, 373 líneas) valida la firma del archivo —porque SharePoint devuelve una página de inicio de sesión con código 200 cuando un enlace deja de ser público—, lee las hojas de superficie, rendimiento y producción, y **busca dinámicamente la fila de encabezados** (la celda «PROVINCIA» en las primeras 40 filas). Esta última decisión surgió de un fallo real: en 2026 la planilla pasó a tener diez filas de título y el lector con posiciones fijas dejó de funcionar.

4.4 Tolerancia a fallos

Las fuentes públicas son, por naturaleza, poco confiables: limitan la tasa de solicitudes, se caen o cambian sin aviso. Se implementó un módulo común de reintentos (`etl/extractors/http_retry.py`) con espera exponencial: hasta seis intentos, con esperas de 2, 4, 8… segundos y un tope de 60 s, que respeta el encabezado `Retry-After` cuando el servidor lo envía. Se reintentan los códigos 429, 500, 502, 503 y 504 y los errores de red transitorios. El Listado 1 muestra el núcleo del algoritmo.

Listado 1. Reintentos con espera exponencial (`etl/extractors/http_retry.py`, fragmento).

```
delay = initial_delay
for attempt in range(1, max_retries + 1):
    last = attempt == max_retries
    try:
        response = client.request(method, url, **kwargs)
    except RETRYABLE_ERRORS:
        if last:
            raise
        time.sleep(delay)
        delay = min(delay * 2, MAX_RETRY_DELAY)
        continue
    if response.status_code not in retry_status or last:
        response.raise_for_status()
        return response
    time.sleep(_retry_after(response, delay))
    delay = min(delay * 2, MAX_RETRY_DELAY)
```

Si una fuente agota los reintentos, el orquestador espera 60 segundos y realiza una **segunda pasada** solo sobre las fuentes caídas. Si vuelve a fallar, utiliza la **última descarga válida** guardada en la capa cruda, registra la situación y continúa: una fuente caída no detiene todo el proceso.

4.5 Almacenamiento de los datos crudos

Cada descarga se guarda como `data/raw/AAAAMMDDhhmmss_<fuente>.<ext>`. Un componente común (`io_utils.py`) construye estos nombres y localiza la descarga más reciente de cada fuente mediante una expresión regular sobre el prefijo. Como ningún archivo se sobrescribe, cada ejecución es reproducible y puede auditarse a posteriori.

4.6 Conclusiones del capítulo

La extracción agrícola transformó dos fuentes pensadas para consulta humana en flujos de datos automatizados. Las lecciones principales son tres: validar siempre el formato de la respuesta (un código HTTP 200 no garantiza datos válidos), no depender de posiciones fijas en documentos que cambian y tratar a cada fuente como potencialmente caída. Los umbrales explícitos (5 % de fallos tolerables, reintentos acotados, respaldo en la última descarga) convierten esas lecciones en comportamientos verificables.

Capítulo 5. Extracción automática de cuadros tarifarios

5.1 El problema

Valorizar la energía exige conocer, para cada lugar del país, el precio al que un usuario compra la electricidad y el precio al que se le reconoce la energía inyectada. Ese dato está disperso en 28 distribuidoras, cada una con su propio sitio, su propio formato y su propia frecuencia de actualización. Algunas publican planillas; otras, documentos PDF con varias tablas lado a lado; otras, páginas construidas con *frameworks* de JavaScript que no contienen enlaces visibles; y algunas, solo imágenes escaneadas. El extractor de tarifas es, por eso, el componente más complejo del sistema: junto con su parser y su registro de distribuidoras suma unas 2.900 líneas, el 59 % del código de extracción.

5.2 Diseño general

La Figura 6 resume el flujo. El diseño separa la **configuración** (qué distribuidoras existen, dónde publican y qué fila buscar) del **código** (cómo descargar, leer e interpretar un documento), de modo que un cambio en un portal se resuelve, en la mayoría de los casos, editando el registro y no la lógica.

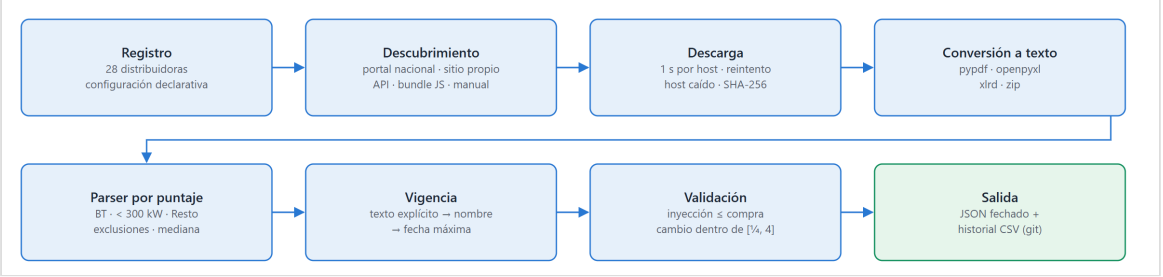


Figura 6. Flujo del extractor de cuadros tarifarios. Fuente: elaboración propia.

Registro declarativo. `etl/reference/distribuidoras.py` describe las 28 distribuidoras: su página en el portal nacional, sus fuentes propias, plantillas de URL por mes y, cuando hace falta, la fila exacta del cuadro que corresponde a la categoría buscada. También define la asignación geográfica: cada provincia se asigna a su distribuidora; Buenos Aires se representa con el promedio de EDEN, EDEA y EDES; y existen excepciones por departamento (por ejemplo, el Gran La Plata corresponde a EDELAP y el este de Mendoza a EDESTE).

Descubrimiento de documentos. Para cada distribuidora se combinan tres tipos de fuente: el portal de la Secretaría de Energía, el sitio propio de la distribuidora o de su ente regulador, y una carpeta de carga manual. El descubrimiento resolvió casos técnicamente diversos:

- **EPE (Santa Fe)**: el sitio, construido con Astro, no publica enlaces; los archivos se obtienen leyendo los identificadores serializados en la página y consultando la API de descarga del propio sitio.
- **EPEC (Córdoba)**: la página es una aplicación Angular; se descarga el *bundle* JavaScript principal y se extraen de él las rutas de los documentos.
- **EJESA (Jujuy)**: los cuadros están en tarjetas sin nombre de archivo ni fecha; se usa el título de la tarjeta (número de resolución) para ordenarlos.
- **APE (La Pampa)**: se recorre el índice anual de resoluciones de la Secretaría de Energía provincial, se excluyen las resoluciones de tarifa mayorista a cooperativas y se leen los anexos dentro de un visor PDF embebido.
- **OCEBA (Buenos Aires)**: un único índice sirve a cuatro distribuidoras, por lo que se descarga una sola vez y se comparte.

Descarga prudente. Se respeta un intervalo mínimo de un segundo por servidor, se identifica al cliente con un *User-Agent* propio y se marca como caído un servidor que falla dos veces seguidas, para no insistir inútilmente. Cada documento se guarda con el nombre de su resumen criptográfico SHA-256, lo que evita duplicados y deja constancia inequívoca de qué documento originó cada valor.

Conversión a texto. Un módulo común detecta el formato real por firma binaria (PDF, XLSX, XLS o ZIP) y convierte cualquier documento a líneas de texto con celdas separadas, de modo que el parser trata igual una planilla que un PDF. Para los cuadros apaisados con varias tablas lado a lado se usa la extracción de PDF que respeta la posición del texto.

5.3 Parser por puntaje

La alternativa de escribir un lector a medida para cada formato fue descartada por su costo de mantenimiento. En su lugar se implementó un **parser heurístico por puntaje** (`tariff_parser.py`) que, sobre cualquier cuadro, busca la tarifa representativa definida en `POL-TARIFF-001`: usuario de demandas medianas o grandes, **baja tensión**, menos de 300 kW, banda horaria «resto», sin impuestos y cuadro pleno. Cada línea candidata recibe puntos según las palabras clave de su contexto (por ejemplo, +3 por «grandes demandas» o «T3», +1 por la banda «resto», −2 por categorías de más de 300 kW) y se descartan de plano las líneas de categorías que no corresponden (residencial, tarifa social, alumbrado público, media y alta tensión, peaje, inyección). Solo se aceptan importes expresados en $/kWh dentro de un rango plausible (30 a 2.000 $/kWh), y el valor final es la **mediana** de los importes de la mejor candidata, robusta frente a columnas subsidiadas (Huber y Ronchetti, 2009). El Listado 2 muestra el núcleo de la selección.

Listado 2. Selección de candidatas en el parser de tarifas (`etl/extractors/tariff_parser.py`, fragmento).

```
# Un PP30 se conecta en baja tensión.
if _voltage_level(lines, i) in {"media", "alta"}:
    continue
score = _category_score(context)
if score == 0:
    continue
raw_values = line_kwh_values(line)
if _bt_mt_columns(lines, i) and len(raw_values) >= 2 and len(raw_values) % 2 == 0:
    raw_values = raw_values[: len(raw_values) // 2]
values = _plausible(raw_values)
if not values:
    continue
if RESTO.search(_label(lines, i)):
    score += 1
if BIG_USER.search(nearby):
    score -= 2
cands.append(Candidate(i, score, values, line.strip()))
```

Para los cuadros en los que la heurística resulta ambigua, el registro permite fijar la fila exacta («ancla» y desplazamiento), como excepción documentada. Por ejemplo, en EDERSA la tarifa de compra es la suma de dos cargos de la categoría T2 y en APE la de dos conceptos de la categoría T2I.

5.4 Determinación de la vigencia

Saber desde cuándo rige un cuadro es tan importante como leer su importe. La vigencia se determina por orden de prioridad: (1) una frase explícita en el texto («vigencia a partir del…»); (2) una fecha en el nombre del archivo o de la carpeta; (3) la fecha más alta en el encabezado del documento. Esta lógica debió contemplar trampas reales: las carpetas de OCEBA con el formato `ctAAAAMMDD`, los cuadros de EJESA con columnas de meses abreviados («Sept-26») y, sobre todo, los **números de expediente** de ocho dígitos que se confundían con fechas (Listado 3).

Listado 3. Fechas de ocho dígitos frente a números de expediente (`etl/extractors/scrape_tarifas.py`, fragmento).

```
# Fechas de 8 dígitos: DDMMAAAA (EPEC) y AAAAMMDD (EPEC hasta 2024).
# Años acotados: los números de expediente de OCEBA
# ("IF-2025-20821026") también tienen 8 dígitos.
for m in re.finditer(r"(?<!\d)(\d{8})(?!\d)", name):
    token = m.group(1)
    d, mo, y = int(token[:2]), int(token[2:4]), int(token[4:])
    if 1 <= d <= 31 and YEAR_MIN <= y <= YEAR_MAX:
        candidates.append(_ym(y, mo) or "")
    y2, mo2, d2 = int(token[:4]), int(token[4:6]), int(token[6:])
    if 1 <= d2 <= 31 and YEAR_MIN <= y2 <= YEAR_MAX:
        candidates.append(_ym(y2, mo2) or "")
```

5.5 Validación y estados

Cada valor extraído se somete a dos reglas: la tarifa de inyección no puede superar la de compra (con una tolerancia de una milésima para redondeos) y la variación respecto del valor anterior debe estar entre un cuarto y cuatro veces. Un valor que viola estas reglas se rechaza y queda registrado. Cada campo de cada distribuidora termina en uno de tres estados: `extraido` (leído en esta corrida), `fallback_previous` (se conserva el último valor extraído válido) o `sin_dato` (nunca se pudo leer). Por decisión explícita, **no se usan valores cargados a mano sin respaldo documental**: una distribuidora sin cuadro legible queda sin dato antes que con un número inventado. La extracción de tarifas nunca detiene el ETL: si falla, se utiliza el último JSON válido.

5.6 Tarifa de inyección faltante

Quince distribuidoras publican su tarifa de inyección en un formato legible. Para las restantes, `POL-TARIFF-002` establece estimarla como la tarifa de compra multiplicada por la **mediana del cociente inyección/compra** observado en las que sí la publican (0,913 con los cuadros de noviembre de 2026), y marcar el valor como estimado. La justificación es normativa: la reglamentación de la Ley 27.424 remunera la energía inyectada a un precio que excluye el valor agregado de distribución, por lo que la inyección es, por construcción, menor o igual que la compra. Como el beneficio es proporcional a 0,6 + 0,4 × cociente, el error de la estimación queda acotado entre −12 % y +4 % para los registros afectados. El tablero permite, además, comparar un escenario conservador (inyección nula), el central (estimada) y uno alto (igual a la compra).

5.7 Historial tarifario

Cada documento procesado se agrega a `data/reference/tarifas_historial.csv`, un archivo de solo agregado que se deduplica por distribuidora y SHA-256: reprocesar un mismo documento no agrega filas. Se eligió CSV en lugar de Parquet para que el historial sea legible y sus cambios aparezcan como *diffs* claros en Git. A noviembre de 2026 el historial contiene 933 filas con vigencias entre julio de 2024 y noviembre de 2026.

5.8 Estado de la extracción

En la ejecución del 11 de noviembre de 2026, **25 de las 28 distribuidoras** se extrajeron automáticamente (11 desde el portal nacional y 14 desde fuentes propias). Las tres restantes carecen de un documento legible por máquina: SPSE (Santa Cruz) publica enlaces rotos, y DPE (Tierra del Fuego) y EDET (Tucumán) publican documentos escaneados. Chubut no tiene una distribuidora provincial única. En conjunto, esas provincias representan el 0,7 % de la energía estimada. La Figura 7 compara las tarifas de compra e inyección de las distribuidoras asignadas a departamentos con producción agrícola.

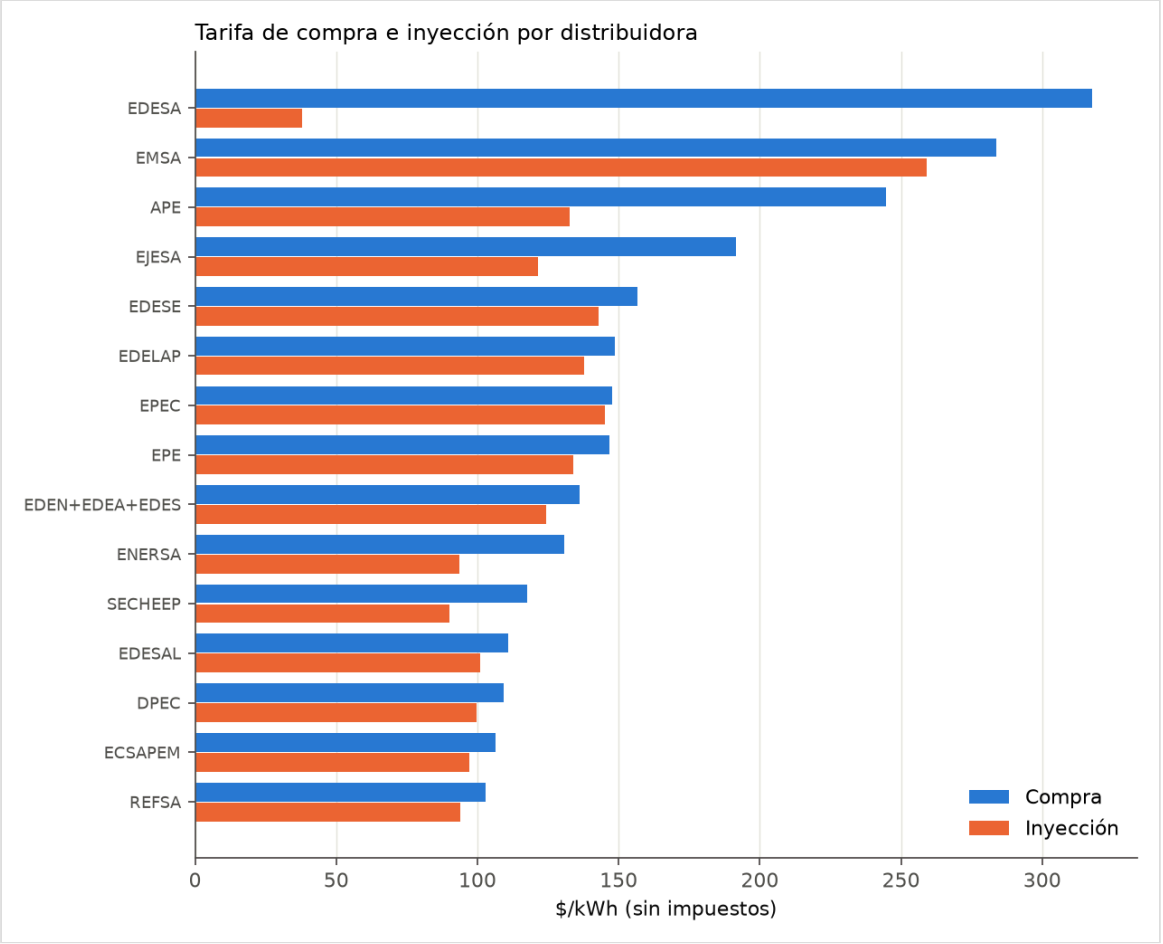


Figura 7. Tarifas de compra e inyección por distribuidora (noviembre de 2026, $/kWh sin impuestos). Fuente: elaboración propia con datos extraídos de los cuadros tarifarios oficiales. Nota: «EDEN+EDEA+EDES» es el promedio utilizado para la provincia de Buenos Aires.

5.9 Conclusiones del capítulo

La extracción tarifaria demostró que es posible obtener de manera automática y trazable un dato que hasta ahora requería un relevamiento manual de 28 sitios. Las claves fueron separar configuración y código, unificar todos los formatos en una representación textual común, reemplazar los lectores a medida por un parser por puntaje con excepciones documentadas, y tratar la vigencia y la validación como problemas de primer orden. Los límites también son claros: sin reconocimiento óptico de caracteres, los cuadros escaneados quedan fuera del alcance automático.

Capítulo 6. Transformación de los datos y modelo de cálculo

Este capítulo describe cómo los datos crudos se convierten en un conjunto consistente y cómo, sobre él, se calculan biomasa, energía y dinero. El orquestador `etl/pipelines/run_etl.py` encadena las etapas en el orden en que se presentan.

6.1 Limpieza

La limpieza (`source_cleaning.py`) resolvió problemas que se descubrieron al contrastar los resultados con las fuentes:

- **Duplicación por producto cartesiano.** Al combinar las hojas de la planilla de Córdoba, las variantes de un mismo cultivo («grano limpio y seco») multiplicaban las filas, y la producción de algunos departamentos aparecía duplicada o cuadruplicada. Se eliminan esos artefactos antes de agrupar.
- **Faltantes que no son ceros.** El MAGyP marca con «SD» (sin dato) los valores desconocidos. Una agregación ingenua los convertía en 0, lo que equivale a afirmar que no hubo producción. La regla `POL-MISSING-001` establece que un faltante se conserva como faltante (`sum(min_count=1)` en pandas) y que los registros no se eliminan, sino que se marcan.
- **Esquema canónico.** Ambas fuentes se llevan a un esquema común: provincia, departamento, campaña, cultivo, superficie (ha), producción (t), rendimiento (t/ha) y fuente.

6.2 Normalización de nombres

Para que «Gral. San Martín», «GENERAL SAN MARTIN» y «general san martín» se reconozcan como el mismo departamento, se aplica una normalización en cuatro niveles (`string_utils.py` y `geography.py`): (1) léxica (minúsculas, sin tildes ni eñes, espacios colapsados); (2) eliminación de puntos; (3) expansión de abreviaturas seguras palabra por palabra (*gral* → *general*, *pte* → *presidente*, *cnl* → *coronel*, etc.); y (4) alias explícitos acotados por provincia (`POL-GEO-001`), como «J. F. Ibarra» → «Juan Felipe Ibarra» en Santiago del Estero. Los alias se limitan a la provincia para no fusionar por error departamentos homónimos de provincias distintas.

Los cultivos se normalizan con una taxonomía de 49 entradas (`crop_taxonomy.py`) que asigna a cada nombre crudo un nombre canónico e indica si es un agregado. `POL-CROP-001` establece que, si para una misma clave coexisten un total («soja total») y sus componentes

(«soja 1.ª», «soja 2.ª»), prevalece el total, para no contar dos veces. La taxonomía falla de inmediato ante un cultivo desconocido (*fail-fast*), en lugar de descartarlo en silencio.

6.3 Reconciliación de fuentes

Cuando el MAGyP y la Bolsa de Córdoba informan el mismo departamento, cultivo y campaña, hay que elegir. `POL-SOURCE-001` establece conservar el registro de **mayor producción** y, ante empate, el del MAGyP. El criterio se fundamenta en que la producción declarada tiende a subestimarse (los registros oficiales provienen de declaraciones juradas) y en que el mayor valor es, en general, el de la fuente con mejor cobertura del departamento. La implementación usa un ordenamiento estable para que el resultado no dependa del orden de llegada de los datos (Listado 4). Los registros descartados (3.298 en la última corrida) y las discrepancias relativas entre fuentes se guardan en la capa de auditoría.

Listado 4. Reconciliación por mayor producción (`etl/transformers/reconciliation.py`, fragmento).

```
sort_cols = key + [prod_col, _PRIORITY_COL]
ascending = [True] * len(key) + [False, True]
work = df.copy()
work[_PRIORITY_COL] = work[FUENTE_COL].map(SOURCE_PRIORITY).fillna(99)
df_sorted = work.sort_values(
    sort_cols, ascending=ascending, na_position="last", kind="mergesort"
)
is_dup = df_sorted.duplicated(subset=key, keep="first")
df_final = df_sorted[~is_dup].drop(columns=[_PRIORITY_COL]).reset_index(drop=True)
```

6.4 Biomasa residual

La biomasa residual en base seca se calcula como:

> **Biomasa (t MS) = P × MS × FR**, con **FR = (1 − IC) / IC**

donde *P* es la producción de grano (t), *MS* la fracción de materia seca del producto y *IC* el índice de cosecha del cultivo. Los coeficientes se toman de `data/reference/ic_fr.csv`, que documenta la fuente bibliográfica de cada valor (Tabla 5). El factor *FR* se recalcula siempre a partir de *IC* para evitar inconsistencias entre columnas.

Tabla 5. Coeficientes agronómicos de los principales cultivos.

Cultivo	IC	FR	MS	Fuente del IC
Maíz	0,47	1,128	0,855	Bolsa de Cereales de Buenos Aires (2015)

Cultivo	IC	FR	MS	Fuente del IC
Soja	0,45	1,222	0,865	Bolsa de Cereales de Buenos Aires (2015)
Trigo	0,35	1,857	0,860	Bolsa de Cereales de Buenos Aires (2015)
Girasol	0,37	1,703	0,890	Raposo (2024)
Cebada	0,53	0,887	0,880	Cuda (2014)
Sorgo	0,35	1,857	0,850	Zamer y Chiarelli
Maní	0,40	1,500	0,900	Haro (2022)
Algodón	0,30	2,333	0,920	Estudios regionales
Arroz	0,45	1,222	0,860	Hay (1995)
Avena	0,35	1,857	0,860	Zubiaga et al. (2023)

Fuente: `data/reference/ic_fr.csv` (17 cultivos en total; se muestran los diez de mayor aporte).

Un registro se considera **válido** para el cálculo solo si su cultivo tiene coeficientes, su superficie es positiva y su residuo es un número finito. Los inválidos no se borran: se marcan con indicadores (`_tiene_ic`, `_rendimiento_invalido`, `_valido_biomasa`) y se informa el impacto de la exclusión en producción y superficie.

6.5 Residuo recuperable

Del total se considera retirable una fracción f = **0,5** (`POL-RECOVERY-001`), valor que la literatura sitúa en el extremo superior de lo sostenible (Wilhelm et al., 2007; Blanco-Canqui y Lal, 2009). Se trata deliberadamente de una cota superior, que el tablero permite reducir hasta el 20 %.

6.6 Energía

La energía se calcula con los parámetros de la ficha técnica del PP30 (`POL-ENERGY-001`):

> **kWh = masa seca (kg) / CE**, con **CE = 1,0 kg MS/kWh**
>
> **Masa a cargar (kg) = masa seca / (1 − H)**, con **H = 12 %**
>
> **Equipos PP30 = kWh / (22 kW × 8.760 h × 0,8)**

La humedad solo determina cuánto material húmedo hay que cargar, pero no altera la energía, porque el consumo específico ya está expresado en base seca. Esta distinción corrigió un error de la primera versión, en la que la humedad se descontaba dos veces. El número de

equipos supone una disponibilidad de 80 % del año y solo tiene sentido sobre cantidades anuales.

6.7 Valorización económica

Bajo el balance neto de la Ley 27.424, una fracción α = **60 %** de la energía se autoconsume y el resto se inyecta:

> **Ahorro = kWh × α × T_compra**
>
> **Crédito = kWh × (1 − α) × T_inyección**
>
> **Beneficio bruto = Ahorro + Crédito**

La tarifa de cada registro se asigna con un resolutor que aplica, en orden, las excepciones por departamento, la distribuidora de la provincia (o el promedio si hay varias) y, si la tarifa de inyección falta, la estimación de `POL-TARIFF-002`. Cada registro conserva el origen de su tarifa (`extraido`, `fallback_previous` o `sin_dato`) y si la inyección fue estimada. El Listado 5 muestra el cálculo vectorizado de energía y dinero.

Listado 5. Energía del PP30 y balance neto (`etl/transformers/energy_finance.py`, fragmentos).

```
residuo = pd.to_numeric(out[residuo_col], errors="coerce")
out[COL_MASA_SECA_KG] = residuo * KG_PER_TON
out[COL_MASA_ALIM_KG] = out[COL_MASA_SECA_KG] / (1.0 - h / 100.0)
out[COL_KWH_TOTALES] = out[COL_MASA_SECA_KG] / ce
out[COL_HORAS] = out[COL_KWH_TOTALES] / pnom
out[COL_EQUIPOS] = out[COL_HORAS] / (HOURS_PER_YEAR * disponibilidad)
# ... valorización (compute_finance_metrics)
out[COL_KWH_AUTO] = kwh * alpha
out[COL_KWH_INY] = kwh * (1.0 - alpha)
out[COL_AHORRO] = out[COL_KWH_AUTO] * out[COL_TARIFA_COMPRA]
out[COL_CREDITO] = out[COL_KWH_INY] * out[COL_TARIFA_INY]
out[COL_BENEFICIO] = out[COL_AHORRO] + out[COL_CREDITO]
```

6.8 Anualización y ejemplo trazable

La producción agrícola varía mucho entre campañas por razones climáticas (la sequía de 2022/23 es el caso extremo reciente). Para no basar la estimación en un solo año, `POL-PERIOD-001` expresa los resultados como **promedio de las cinco últimas campañas completas**. Una campaña se considera incompleta si tiene menos del 80 % de los registros de la mediana de las anteriores, lo que excluye automáticamente la campaña en curso. El promedio se calcula como la suma del período dividida por cinco, de modo que un

departamento sin registro en una campaña cuenta como cero, en lugar de inflar el promedio con solo los años buenos. Tras anualizar se recalculan la energía y el dinero, para que el número de equipos quede expresado por año.

Ejemplo trazable. Para 10.000 t de maíz en un departamento bonaerense: biomasa = 10.000 × 0,855 × 1,128 ≈ 9.641 t MS; recuperable = 4.821 t MS; energía ≈ 4,82 GWh; masa a cargar ≈ 5.478 t; equipos ≈ 4.821.000 / 154.176 ≈ 31; ahorro ≈ 4,82 GWh × 0,6 × 135 $/kWh ≈ $390 millones; crédito ≈ 4,82 GWh × 0,4 × 135 × 0,917 ≈ $239 millones; beneficio bruto ≈ **$629 millones por año**. El sistema reproduce este resultado, y el caso está codificado como prueba automatizada.

6.9 Persistencia

Los resultados se escriben en Parquet con compresión Snappy: `biomasa_validos.parquet` (110.819 registros históricos, 33 columnas) y `potencial_energetico_anual.parquet` (2.421 registros por provincia, departamento y cultivo, 25 columnas). Las columnas y su significado están documentadas como **contratos de artefactos** en `docs/03_datos/contratos_de_artefactos.md`, de modo que el tablero y cualquier consumidor futuro tienen una especificación estable.

6.10 Conclusiones del capítulo

La transformación combina reglas de limpieza descubiertas empíricamente con un modelo de cálculo deliberadamente simple. Cada paso del modelo tiene una justificación bibliográfica o normativa y un identificador de decisión. Dos principios atraviesan el capítulo: no inventar datos (un faltante es un faltante, una tarifa ilegible queda sin dato) y no ocultar exclusiones (todo lo descartado queda disponible para auditoría). La linealidad del modelo, lejos de ser una limitación, es lo que permite que el tablero recalcule escenarios al instante.

Capítulo 7. Calidad del software y de los datos

La calidad se abordó en dos planos complementarios: la **calidad del código**, verificada antes de integrar cada cambio, y la **calidad de los datos**, verificada en cada ejecución del ETL (`POL-QA-001`).

7.1 Pruebas automatizadas

El repositorio contiene **120 funciones de prueba** en 12 archivos, que, al incluir los casos parametrizados, ejecutan **156 casos**. La suite completa se ejecuta en unos cuatro segundos, sin acceso a la red, gracias a respuestas grabadas y datos de ejemplo. En la versión entregada, los 156 casos pasan.

Tabla 6. Distribución de las pruebas por módulo.

Archivo	Pruebas	Qué verifica
`test_energy_tarifas.py`	32	Energía del PP30, asignación de tarifas, período de referencia
`test_tariff_parser.py`	29	Lectura de cuadros reales de cada distribuidora
`test_etl_fixes.py`	16	Regresiones de errores corregidos
`test_dashboard.py`	8	Carga de datos, cruce con GADM, escenarios
`test_crop_consolidation.py`	6	Regla de totales y componentes
`test_notifications.py`	6	Avisos por mensaje
`test_data_checks.py`	5	Alertas de calidad y modo estricto
`test_geography.py`	5	Normalización y alias de departamentos
`test_biomass.py`	4	Fórmula de biomasa e indicadores de validez
`test_string_utils.py`	4	Normalización de texto
`test_reconciliation.py`	3	Elección por mayor producción
`test_validation.py`	2	Invariantes

Fuente: elaboración propia.

Los nombres de las pruebas funcionan como especificación ejecutable del comportamiento esperado. Algunos ejemplos son `test_cadena_energetica_no_descuenta_humedad_dos_veces`, `test_magyp_sd_no_se_convierte_en_cero`, `test_cba_merge_cartesiano_no_multiplica_produccion`, `test_empate_prefiere_magyp_independiente_del_orden`, `test_annualize_divide_por_n_campanas` y `test_un_aviso_fallido_no_rompe_el_etl`.

7.2 Invariantes

Las invariantes son propiedades que deben cumplirse siempre; si alguna se rompe, el ETL se detiene **antes de publicar**. Se verifican, entre otras: que tras reconciliar no haya claves duplicadas; que el conjunto de claves de salida sea igual al de entrada; que el registro elegido tenga la producción máxima de su clave; que superficies y producciones no sean negativas; y que la biomasa válida no contenga infinitos ni valores negativos. Se implementan con excepciones explícitas y no con la sentencia `assert` de Python, que se desactiva cuando el intérprete se ejecuta con optimizaciones.

7.3 Alertas de calidad

A diferencia de una invariante, una **alerta** señala algo sospechoso pero posible. Cada ejecución compara sus resultados con los de la anterior:

Tabla 7. Controles de calidad que generan alertas.

Control	Se dispara cuando	Umbral
`salto_tarifa`	La tarifa de compra de una distribuidora cambia respecto del cuadro anterior	> 50 %
`cambio_total`	La energía o el beneficio total cambian respecto de la corrida anterior	> 20 %
`cambio_provincia`	La energía de una provincia relevante (≥ 1 % del total) cambia	> 50 %

Fuente: `etl/transformers/data_checks.py`.

Las alertas se registran en el log, se guardan en `data/audit/alertas_calidad.parquet` y se muestran en el tablero. Con la opción `--estricto`, una alerta también detiene la publicación. El primer hallazgo de este mecanismo fue real: un salto de la tarifa de ENERSA de 67,28 a 130,84 $/kWh reveló que el valor anterior provenía, por error, de una tabla de media tensión impresa junto a la de baja tensión.

7.4 Avisos y detección de datos desactualizados

Un sistema automático que falla en silencio es peor que uno manual. Por eso el ETL envía avisos (`etl/notifications.py`) por Telegram o por cualquier *webhook* compatible (Slack, Discord, Teams, Google Chat) cuando una ejecución falla o produce alertas, y opcionalmente cuando termina sin problemas. Tres decisiones de diseño merecen destacarse: un aviso que falla nunca hace fallar el ETL; las credenciales (URL del *webhook*, *token* del bot) se leen de variables de entorno y nunca se escriben en el log; y, desde noviembre de 2026, solo se notifican las alertas **nuevas**, identificadas mediante una clave compuesta por control, entidad y valores, para evitar repetir semanalmente el mismo aviso. Como complemento, el

tablero muestra un cartel de advertencia si los datos tienen más de 192 horas (una ejecución semanal perdida más un día de margen), lo que cubre el caso en que el ETL ni siquiera llega a ejecutarse.

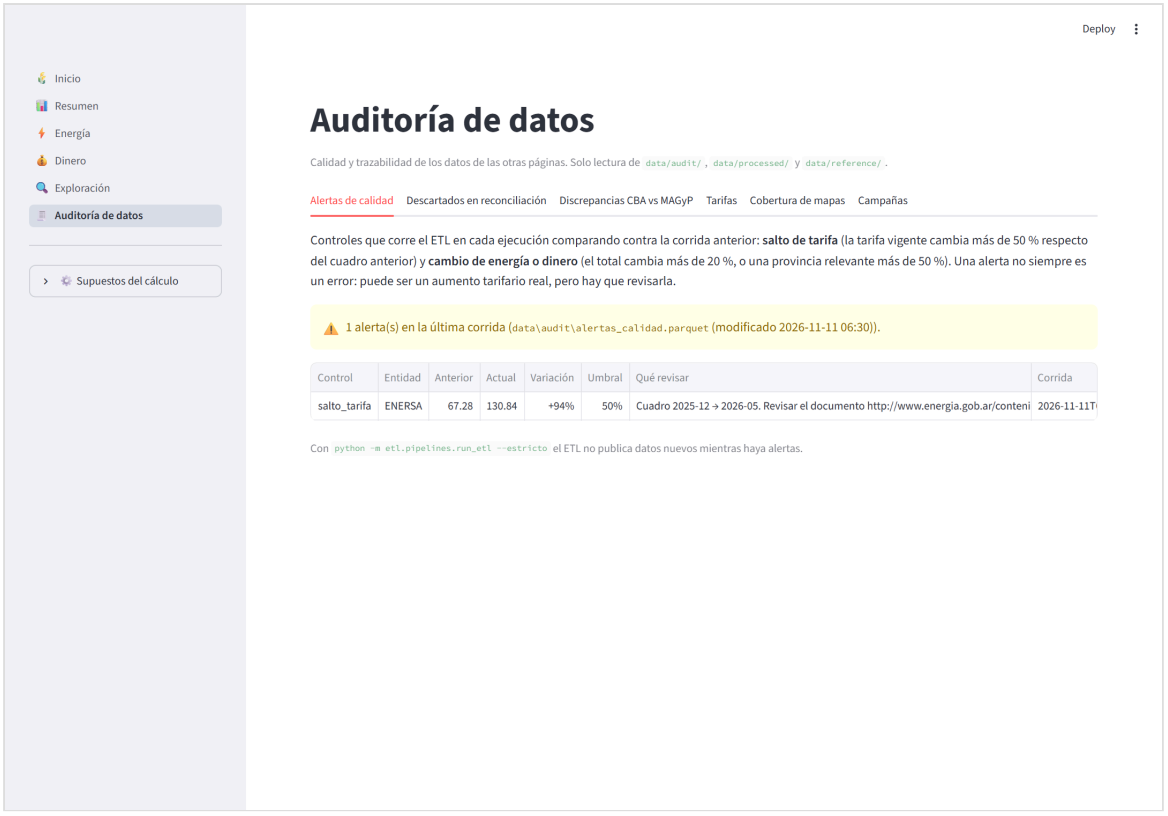


Figura 8. Página «Auditoría de datos» del tablero, con alertas, registros descartados, discrepancias entre fuentes, estado de las tarifas y cobertura de mapas. Fuente: captura del sistema.

7.5 Errores detectados y lecciones

La fase de validación contra fuentes reales (Sección 2.2) detectó errores que ilustran por qué la calidad de los datos requiere controles específicos:

Tabla 8. Errores relevantes detectados y su corrección.

Error	Causa	Corrección	Lección
Producción ×2 o ×4 en Córdoba	Producto cartesiano al combinar hojas	Deduplicación previa	Auditar totales contra la fuente
Faltantes convertidos en 0	Agregación ingenua de «SD»	`POL-MISSING-001`	Un faltante no es un cero
Humedad descontada dos veces	Consumo específico mal interpretado	Base seca explícita	Declarar la base de cada magnitud
Tarifa de OCEBA demasiado baja	Se leía la tarifa mayorista a distribuidores municipales	Exclusión de bloques mayoristas	El contexto semántico importa

Error	Causa	Corrección	Lección
«800 $/kWh» en EDEMSA	Se leía una cantidad («hasta 800 kWh») como precio	Rango y unidad obligatorios	Validar unidad y plausibilidad
Expedientes leídos como fechas	Números de 8 dígitos	Años acotados a 2015–2040	Las heurísticas de fechas necesitan rangos
Datos viejos en el tablero	Caché indexada por ruta y no por fecha	Caché por *mtime*	Invalidar la caché por contenido
Alertas repetidas cada semana	Sin memoria de alertas previas	Columna nueva	Notificar cambios, no estados

Fuente: elaboración propia a partir del historial de commits.

7.6 Conclusiones del capítulo

La calidad se construyó en capas: pruebas que protegen el código, invariantes que impiden publicar datos imposibles, alertas que señalan datos sospechosos y avisos que garantizan que alguien se entere. La experiencia mostró que los errores más dañinos no son los que provocan una excepción, sino los **silenciosos**: los que producen un número verosímil pero incorrecto. Contra ellos, la única defensa eficaz fue contrastar con las fuentes y convertir cada hallazgo en una prueba de regresión.

Capítulo 8. Infraestructura, despliegue y operación

8.1 Contenerización

El sistema se empaqueta en una imagen Docker construida en **dos etapas** sobre `python:3.11-slim`. La primera etapa compila todas las dependencias a partir de `pyproject.toml` únicamente, de modo que esa capa —la más pesada, con pandas, PyArrow y Streamlit— se reutiliza mientras las dependencias no cambien, y una modificación del código no obliga a reinstalarlas. La segunda etapa instala esas dependencias, copia el código, crea un usuario sin privilegios y define como comando por defecto la ejecución del ETL. El archivo `.dockerignore` excluye datos, entornos virtuales, *notebooks* y documentación, lo que reduce el tamaño de la imagen y evita filtrar archivos locales.

8.2 Servicios

`docker-compose.yml` declara cuatro servicios que comparten la misma imagen (Tabla 9).

Tabla 9. Servicios de Docker Compose.

Servicio	Función	Volúmenes
`etl_worker`	Contenedor permanente donde el planificador ejecuta el ETL	`data`, `config` (solo lectura), `logs`
`etl_pipeline`	Ejecución manual única (perfil `etl`)	Los mismos
`scheduler`	Planificador Ofelia	*Socket* de Docker (solo lectura), `schedule.ini`
`dashboard`	Tablero Streamlit en el puerto 8501	`data` en solo lectura

Fuente: `docker-compose.yml`.

El tablero monta los datos en **solo lectura**: aunque fuera comprometido, no podría alterar los resultados. Las credenciales de los avisos se inyectan como variables de entorno, conforme al principio de separar configuración y código (Wiggins, 2011).

8.3 Planificación

El planificador **Ofelia** ejecuta el ETL dentro del contenedor trabajador según una expresión *cron* de seis campos (con segundos) (Listado 6). Inicialmente la ejecución era diaria; en noviembre de 2026 se pasó a **semanal** (lunes a las 06:00), porque las tarifas cambian como máximo una vez por mes y las estimaciones agrícolas con menor frecuencia aún, mientras

que cada ejecución insume unos 30 minutos de descargas: ejecutar a diario solo generaba carga sobre servidores públicos para repetir los mismos datos.

Listado 6. Programación semanal del ETL (`config/schedule.ini`).

```
[job-exec "etl-biomasa"]
# Semanal: las tarifas cambian una vez por mes como máximo.
schedule = 0 0 6 * * 1
container = biomasa_etl_worker
command = python -m etl.pipelines.run_etl --config config/paths.yaml
```

8.4 Servidor de producción

El sistema se opera en un **servidor privado virtual** con Debian (2 vCPU y 8 GB de RAM), cuya topología se muestra en la Figura 9. El diseño aplica el principio de **mínima exposición**:

- **Cortafuegos (`ufw`)**: solo se aceptan conexiones entrantes a los puertos 22 (SSH), 80 y 443 (web), además del tráfico de la interfaz de la VPN.
- **Proxy inverso con TLS**: **Caddy** recibe las conexiones HTTPS, obtiene y renueva automáticamente el certificado y reenvía las solicitudes al tablero, que escucha solo en la red interna (puerto 8501). El usuario nunca se conecta directamente al contenedor.
- **Aplicación**: el *stack* de Docker Compose descripto en la Sección 8.2, con los datos en un volumen persistente.

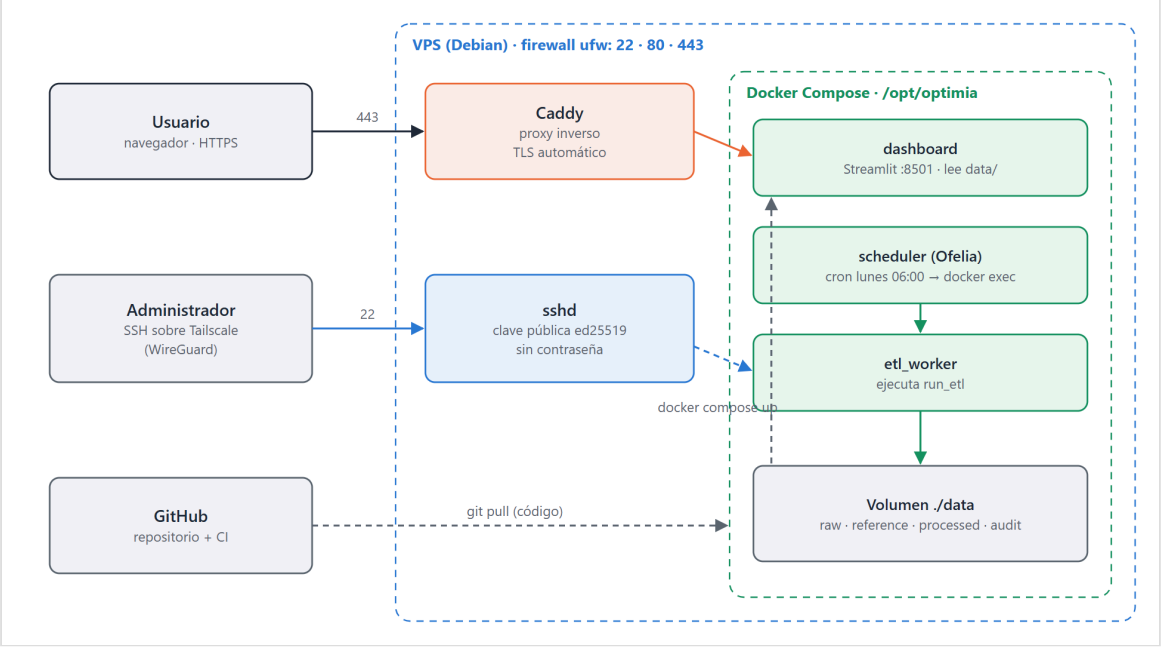


Figura 9. Topología de despliegue en el VPS. Fuente: elaboración propia.

8.5 SSH aplicado a la operación

Los conceptos de la Sección 1.7 se aplican en la operación del servidor de la siguiente manera:

1. **Autenticación exclusivamente por clave pública.** El administrador se conecta con un par de claves Ed25519: la clave privada permanece en su equipo y la pública está registrada en `authorized_keys` del servidor. Al no viajar ninguna contraseña, los intentos de fuerza bruta que reciben constantemente los servidores expuestos a Internet resultan ineficaces.
2. **Verificación del servidor.** En la primera conexión el cliente registra la *host key* del VPS en `known_hosts`; en adelante, cualquier cambio inesperado de esa clave se advierte como un posible ataque de intermediario.
3. **Acceso a través de una VPN.** El acceso administrativo se realiza sobre Tailscale (basado en WireGuard), cuya interfaz está habilitada en el cortafuegos. Esto permite, llegado el caso, cerrar el puerto 22 a Internet sin perder la capacidad de administrar el servidor.
4. **Claves de despliegue con comando forzado.** Para la automatización, el servidor dispone de un usuario de despliegue cuyas claves se registran con un comando forzado y la opción `restrict` (Listado 7). Una clave así no puede abrir una terminal, reenviar puertos ni ejecutar otra cosa que el despliegue del proyecto al que pertenece: aunque fuera robada, el daño posible queda acotado.
5. **Accesos temporales con certificados.** Para sesiones de mantenimiento asistidas se emiten certificados SSH de corta duración (dos horas), firmados por una autoridad certificante propia en la que el servidor confía, de modo que el acceso caduca solo.
6. **Secretos fuera del repositorio.** Las credenciales residen en el servidor con permisos restringidos (`600`) y nunca se versionan.

Listado 7. Registro de una clave de despliegue con comando forzado en `authorized_keys` (valores ilustrativos).

```
command="/opt/ops/bin/deploy-gate <proyecto>",restrict ssh-ed25519 AAAAC3Nza... deploy@<proyecto>
```

En la operación cotidiana, actualizar el sistema consiste en conectarse por SSH al servidor, obtener la última versión del repositorio (`git pull`) y reconstruir los servicios (`docker compose up -d --build`). La ejecución manual del ETL puede dispararse con `docker compose --profile etl run --rm etl_pipeline`.

8.6 Integración continua

Cada *pull request* hacia `main` dispara un flujo de GitHub Actions con tres trabajos (Figura 10):

1. **`tests`**: instala el proyecto, ejecuta un análisis estático acotado a errores reales (`ruff --select E9,F`) y corre la suite de pruebas.
2. **`imagen`**: solo si cambiaron `Dockerfile`, `pyproject.toml` o `.dockerignore`, construye la imagen y verifica que el ETL y el tablero se importen dentro de ella. Construirla siempre habría consumido minutos de cómputo sin aportar información.
3. **`automerge`**: si los dos anteriores pasan, el PR no es borrador ni tiene la etiqueta `no-automerge`, fusiona automáticamente el cambio (Listado 8). La opción `--match-head-commit` impide fusionar si llegó un nuevo *commit* mientras se ejecutaban las pruebas, lo que evita integrar código no verificado.

Listado 8. Fusión automática condicionada a la integración continua (`.github/workflows/ci.yml`, fragmento).

```
automerge:
  needs: [tests, imagen]
  if: >-
    github.event_name == 'pull_request' &&
    github.event.pull_request.base.ref == 'main' &&
    !github.event.pull_request.draft &&
    !contains(github.event.pull_request.labels.*.name, 'no-automerge')
  permissions: { contents: write, pull-requests: write }
  steps:
    - run: gh pr merge "${{ github.event.pull_request.number }}" --merge \
           --match-head-commit "${{ github.event.pull_request.head.sha }}"
```

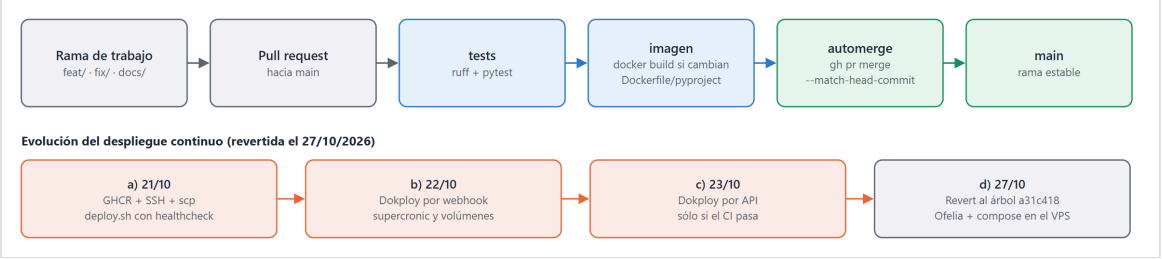


Figura 10. Flujo de integración continua y evolución del despliegue continuo. Fuente: elaboración propia.

8.7 Experiencia con el despliegue continuo

Entre el 21 y el 27 de octubre de 2026 se exploraron tres diseños de **despliegue continuo** (parte inferior de la Figura 10), en el marco de una reestructuración del repositorio en un *monorepo* que unificaba este trabajo con otro proyecto de energía solar:

- **(a) Registro de imágenes y SSH.** El *pipeline* construía la imagen, la publicaba en GitHub Container Registry y, con una clave privada guardada como secreto del repositorio, se conectaba por SSH al servidor, copiaba la configuración con `scp` y ejecutaba un script que actualizaba los servicios y esperaba a que el tablero respondiera a su chequeo de salud.

- **(b) Dokploy por *webhook*.** Se reemplazó el script por una plataforma de despliegue autoalojada y Ofelia por *supercronic*, con volúmenes con nombre y chequeos de salud. Se detectó que el *webhook* desplegaba aun cuando la integración continua fallaba.
- **(c) Dokploy por API.** El despliegue pasó a dispararse por la API de la plataforma, solo tras un CI exitoso sobre `main`.

El 27 de octubre se decidió **revertir** el conjunto (commit `63cbb88`) y volver exactamente al árbol previo a la reestructuración. La reversión eliminó 5.764 líneas en 161 archivos. Las razones fueron de alcance y de riesgo: la unificación con el proyecto solar ampliaba el sistema más allá de los objetivos de esta tesis y la plataforma de despliegue agregaba un componente que administrar sin beneficios proporcionales para un sistema que se actualiza una vez por semana. La experiencia, sin embargo, dejó aprendizajes que se conservan como trabajo futuro (Sección 11.3): la importancia de condicionar el despliegue al éxito de las pruebas, el valor de los chequeos de salud y la conveniencia de restringir las claves de despliegue con comandos forzados.

8.8 Recursos y mantenimiento

El tablero consume entre 300 y 600 MB de memoria y el ETL hasta 1 GB durante su ejecución; la imagen ocupa alrededor de 1 GB. Los datos crecen unos 0,5 GB por año, principalmente por la capa cruda. La documentación de operación incluye guías de instalación, ejecución, solución de problemas ya ocurridos y una lista blanca de los dominios a los que el ETL necesita acceder, útil para configurar un cortafuegos de salida.

8.9 Conclusiones del capítulo

La infraestructura traduce los requerimientos no funcionales en mecanismos concretos: Docker garantiza la reproducibilidad, Ofelia la actualización periódica, el volumen persistente la conservación de los datos, el proxy inverso con TLS el acceso seguro de los usuarios y SSH con autenticación por clave, VPN y comandos forzados el acceso seguro de los administradores. La exploración del despliegue continuo y su posterior reversión ilustran una decisión de ingeniería madura: preferir la solución más simple que satisface los requerimientos y documentar las alternativas evaluadas para retomarlas cuando la escala lo justifique.

Capítulo 9. Tablero de visualización

9.1 Objetivo y diseño

El tablero traduce los resultados del ETL en información comprensible para usuarios no técnicos: productores, técnicos de extensión, cooperativas eléctricas o responsables de política energética. Se construyó con Streamlit y Plotly y se organiza en seis páginas con navegación explícita. Los principios de diseño adoptados fueron: indicadores principales arriba, en forma de cifras destacadas; gráficos de barras horizontales para comparar magnitudes entre categorías; mapas coropléticos con una escala secuencial de un solo tono (Harrower y Brewer, 2003), con una capa base gris que mantiene visibles las jurisdicciones sin dato; y colores consistentes en todas las vistas (azul para autoconsumo, naranja para inyección).

9.2 Páginas

Inicio (Figura 11). Explica en lenguaje simple qué calcula el sistema, la cadena de cálculo y la naturaleza de los resultados (potencial técnico y beneficio bruto). Un desplegable muestra la fecha de actualización de cada archivo de datos.



Figura 11. Página de inicio del tablero. Fuente: captura del sistema.

Resumen (Figura 12). Permite elegir campañas y cultivos, y alternar entre biomasa, energía y dinero. Presenta cuatro indicadores anuales, rankings y mapas por provincia y por departamento.

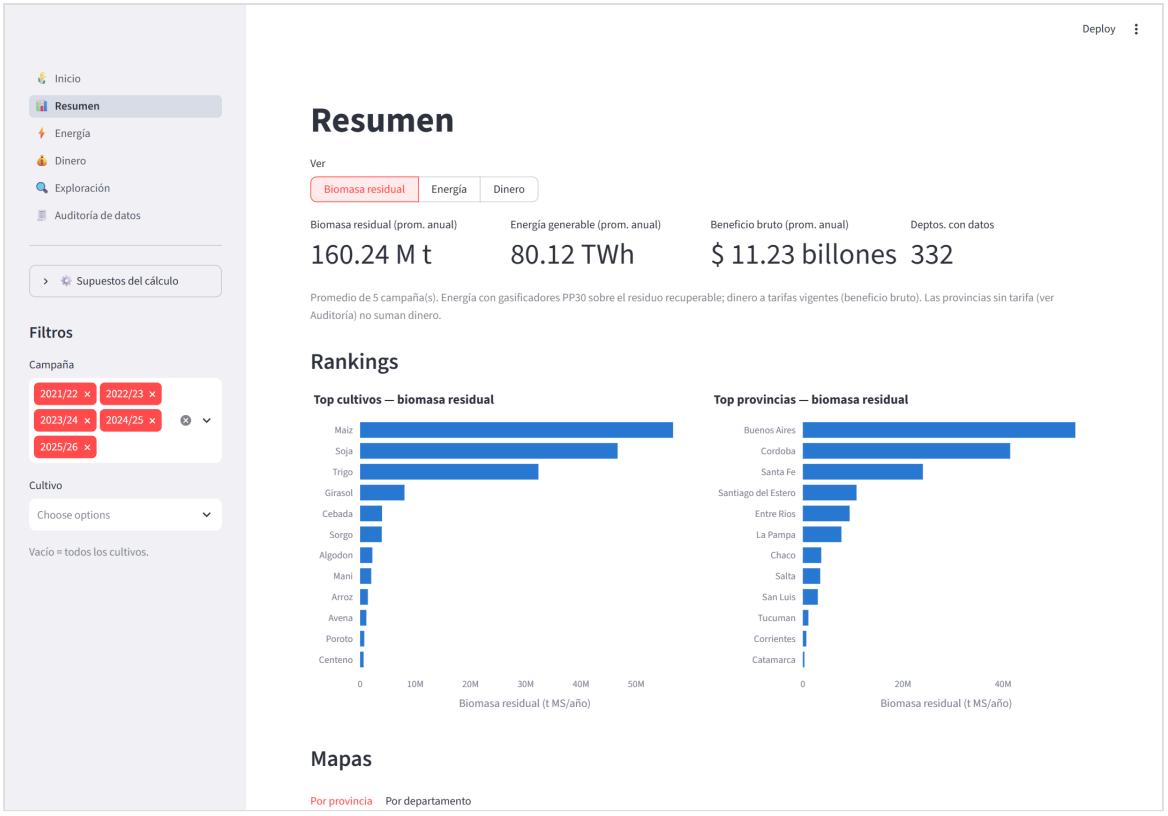


Figura 12. Página «Resumen». Fuente: captura del sistema.

Energía (Figura 13). Muestra la energía generable por año, los equipos PP30 equivalentes y el residuo recuperable, con barras apiladas por provincia (autoconsumo e inyección), barras por cultivo, mapas y un desplegable con los parámetros del método.

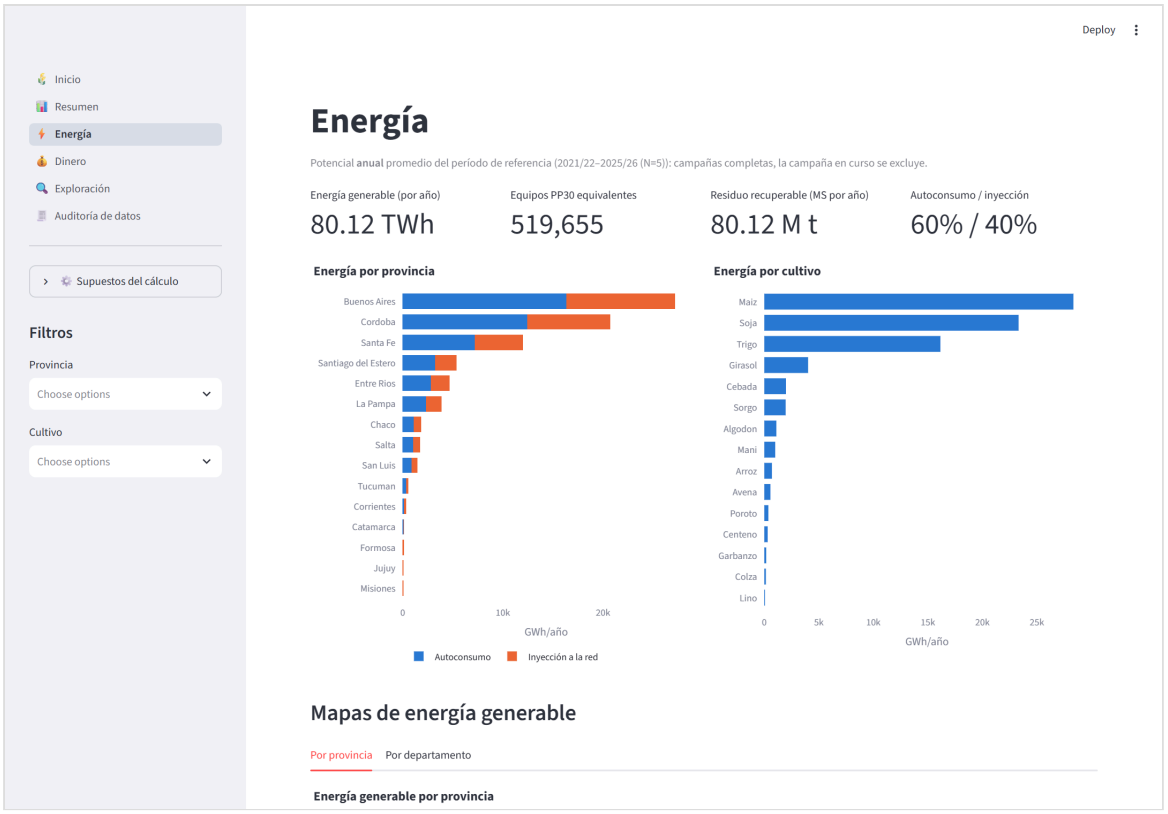


Figura 13. Página «Energía». Fuente: captura del sistema.

Dinero (Figura 14). Presenta el beneficio bruto, el ahorro por autoconsumo, el ingreso por inyección y el porcentaje de energía con tarifa; advierte qué proporción usa una tarifa de inyección estimada; e incluye la tabla de tarifas por distribuidora con su estado y su historial.

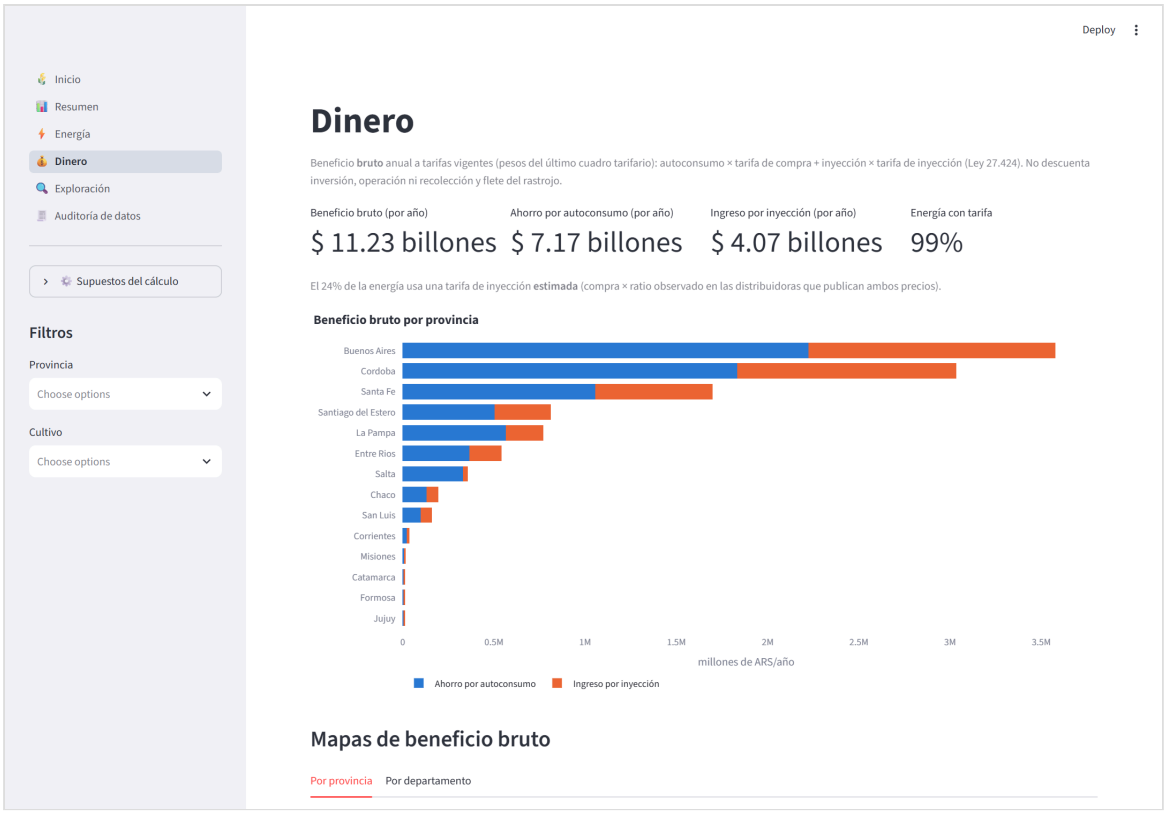


Figura 14. Página «Dinero». Fuente: captura del sistema.

9.3 Supuestos ajustables

El panel lateral «Supuestos del cálculo» (Figura 15) permite modificar los cuatro supuestos más inciertos: el porcentaje de autoconsumo (0–100 %), el escenario de inyección donde no hay precio publicado (conservador, central o alto), el consumo específico del PP30 (1,0 o 1,1 kg MS/kWh) y la fracción de rastrojo retirada (20–70 %). Gracias a la linealidad del modelo, el tablero recalcula energía y dinero multiplicando columnas de los archivos ya calculados, sin volver a ejecutar el ETL. Cuando un supuesto difiere del valor de referencia, el tablero lo advierte de manera visible y ofrece restaurarlo.

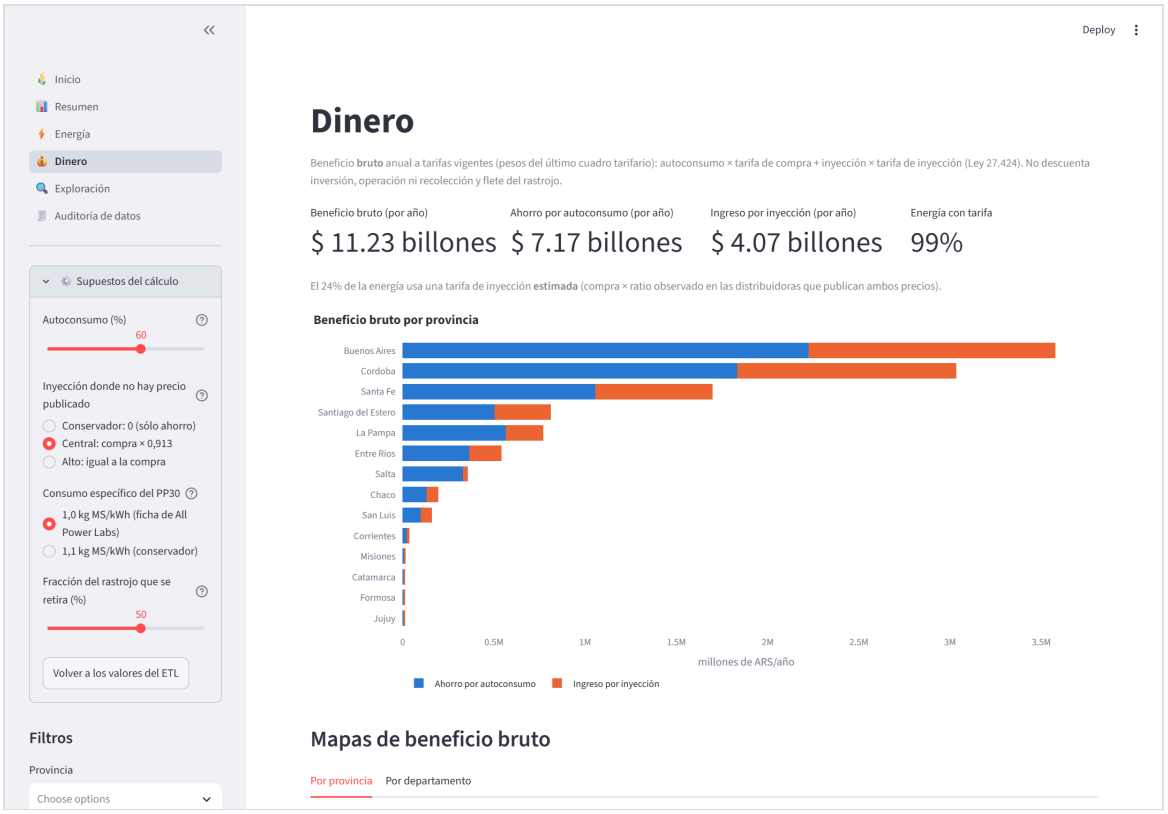


Figura 15. Panel de supuestos ajustables. Fuente: captura del sistema.

Exploración (Figura 16). Filtros encadenados por provincia, departamento, cultivo y campaña; serie temporal por campaña; y una tabla descargable en CSV.

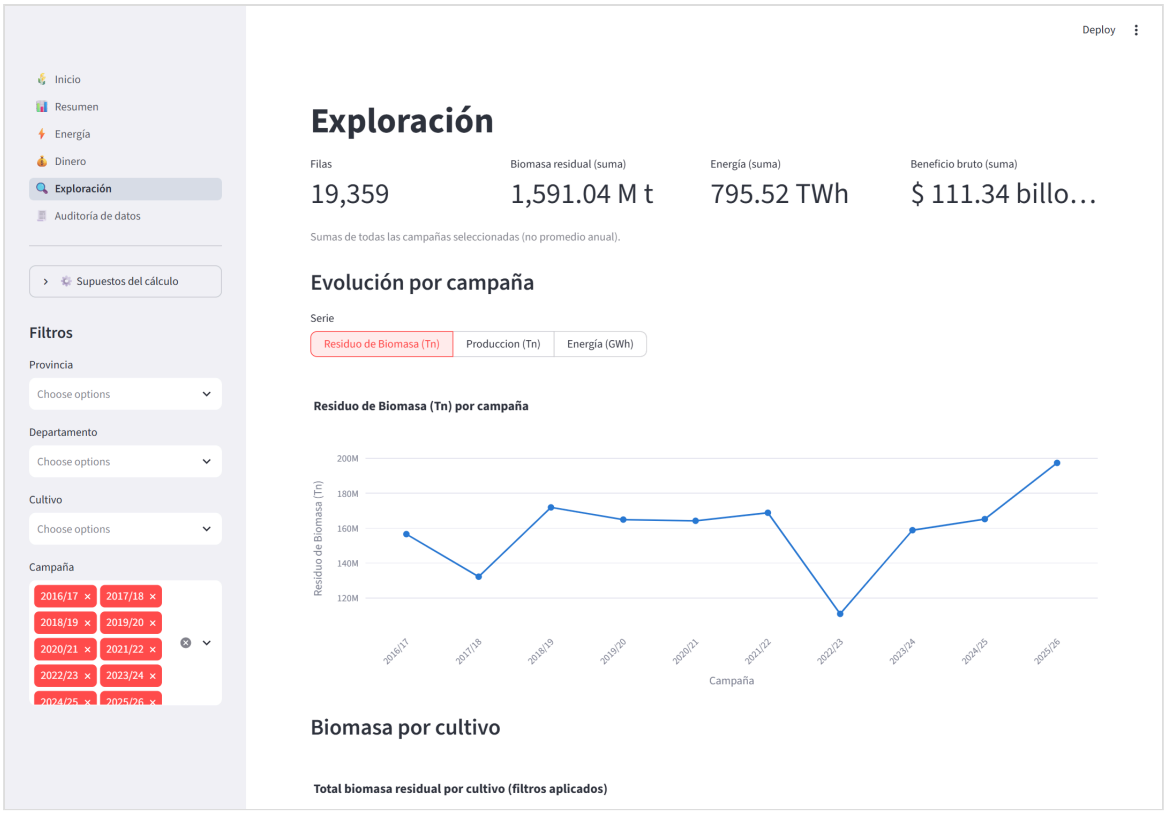


Figura 16. Página «Exploración». Fuente: captura del sistema.

Auditoría de datos (Figura 8, Capítulo 7). Expone las alertas de calidad, los registros descartados en la reconciliación, las discrepancias entre fuentes, el estado de cada tarifa, la cobertura de los mapas y las campañas consideradas.

9.4 Aspectos técnicos

La carga de datos (`dashboard/data_loader.py`) usa una conexión DuckDB en memoria por proceso y una caché cuya clave incluye la fecha de modificación del archivo: cuando el ETL reescribe un Parquet, el tablero lo relee automáticamente sin reiniciarse. La caché se acota a un número máximo de entradas, porque en una versión anterior cada ejecución del ETL dejaba en memoria otra copia de unos 93 MB. El cruce entre los nombres de departamentos y los polígonos de GADM 4.1 requirió una clave normalizada `provincia|departamento`, la conversión de números a letras y un diccionario de nombres renombrados, cuyo funcionamiento está cubierto por pruebas.

9.5 Conclusiones del capítulo

El tablero cierra la cadena de valor del sistema: convierte archivos técnicos en respuestas visuales a las preguntas del problema. Su principal aporte metodológico es el panel de supuestos, que convierte la incertidumbre del modelo en una herramienta de exploración en

lugar de ocultarla. La página de auditoría, por su parte, materializa el principio de transparencia que guió todo el trabajo: cualquier usuario puede ver qué se excluyó y por qué.

Capítulo 10. Resultados

Los resultados corresponden a la ejecución del ETL del 11 de noviembre de 2026, con período de referencia **2021/22–2025/26** (cinco campañas completas) y tarifas sin impuestos vigentes en noviembre de 2026. Todas las cifras son **promedios anuales** del período y representan un **potencial técnico** y un **beneficio bruto**.

10.1 Cobertura de los datos

La base histórica reconciliada contiene **110.819 registros válidos** que abarcan 57 campañas (de 1969/70 a 2025/26). El resultado anual cubre **332 departamentos de 15 provincias** y **15 cultivos**, sobre una superficie cosechada promedio de **38,0 millones de hectáreas** y una producción de **141,7 millones de toneladas** de grano por año. La Figura 17 muestra la evolución de la biomasa residual total, en la que se aprecian la expansión agrícola de las últimas décadas y la caída asociada a la sequía de 2022/23.

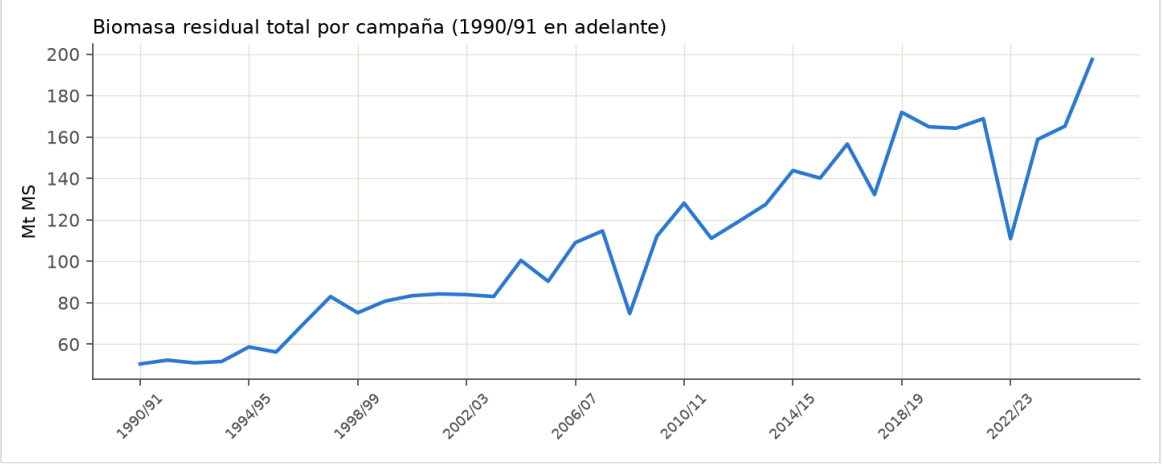


Figura 17. Biomasa residual total por campaña, desde 1990/91. Fuente: elaboración propia con datos del MAGyP y la BCCBA.

10.2 Indicadores principales

Tabla 10. Indicadores nacionales anuales (período 2021/22–2025/26).

Indicador	Valor
Biomasa residual (materia seca)	160,2 Mt/año
Residuo recuperable (f = 50 %)	80,1 Mt/año
Energía eléctrica generable	**80,1 TWh/año**
Energía por hectárea cosechada	≈ 2.107 kWh/ha
Equipos PP30 equivalentes	≈ 519.700 (≈ 11,4 GW)

Indicador	Valor
Ahorro por autoconsumo (60 %)	7,17 billones de $/año
Crédito por inyección (40 %)	4,07 billones de $/año
Beneficio bruto	**11,23 billones de $/año**
Energía con tarifa asignada	99,3 %
Energía con tarifa de inyección estimada	24,3 %

Fuente: elaboración propia.

Para dimensionar la cifra de energía, la demanda local de energía eléctrica del Mercado Eléctrico Mayorista fue de 138.755 GWh en 2022 (CAMMESA, 2023): el potencial técnico estimado equivale a alrededor del **58 %** de esa demanda. El número de equipos —más de medio millón de gasificadores de 22 kW— pone de relieve que se trata de un potencial teórico-técnico y no de un escenario de implementación.

10.3 Distribución territorial

Tabla 11. Resultados anuales por provincia.

Provincia	Distribuidora	Biomasa (Mt MS)	Energía (TWh)	% energía	Equipos PP30	Beneficio (miles de M$)
Buenos Aires	EDEN+EDEA+EDES	54,4	27,21	34,0	176.489	3.577
Córdoba	EPEC	41,4	20,70	25,8	134.268	3.036
Santa Fe	EPE	24,0	11,99	15,0	77.755	1.698
Santiago del Estero	EDESE	10,7	5,36	6,7	34.776	811
Entre Ríos	ENERSA	9,3	4,67	5,8	30.301	542
La Pampa	APE	7,7	3,85	4,8	24.990	770
Chaco	SECHEEP	3,7	1,84	2,3	11.918	196
Salta	EDESA	3,5	1,73	2,2	11.248	357
San Luis	EDESAL	3,0	1,50	1,9	9.706	160
Tucumán	EDET (sin dato)	1,1	0,55	0,7	3.589	—
Otras (5)	—	1,4	0,72	0,9	4.615	88
Total		**160,2**	**80,12**	**100**	**519.655**	**11.232**

Fuente: elaboración propia. «Otras» agrupa Corrientes, Catamarca, Formosa, Jujuy y Misiones.

Buenos Aires, Córdoba y Santa Fe concentran el **74,8 %** de la energía (Figura 18). A escala departamental, los cinco departamentos con mayor potencial son Río Cuarto (2,91 TWh/año), General López (2,37), Marcos Juárez (2,36), Unión (2,26) y General Roca (2,10), cuatro de ellos en Córdoba (Figura 20). La concentración territorial sugiere que una eventual política de aprovechamiento debería focalizarse en el núcleo pampeano.

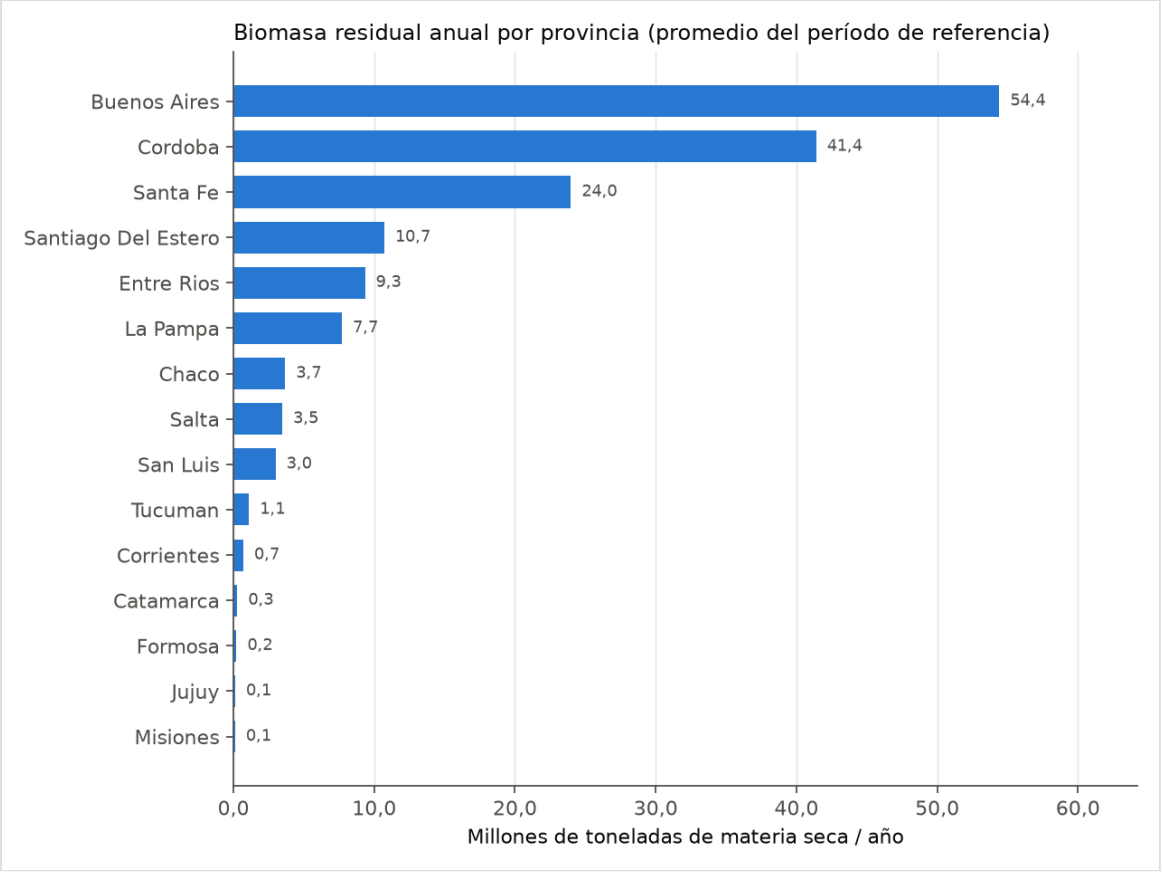


Figura 18. Biomasa residual anual por provincia. Fuente: elaboración propia.

10.4 Distribución por cultivo

Maíz (35,4 %), soja (29,1 %) y trigo (20,2 %) explican el **84,6 %** de la energía; les siguen girasol (5,0 %) y cebada (2,5 %) (Figura 19). El trigo tiene un peso mayor que su participación en la producción porque su índice de cosecha es bajo (0,35) y deja, por lo tanto, más residuo por tonelada de grano.

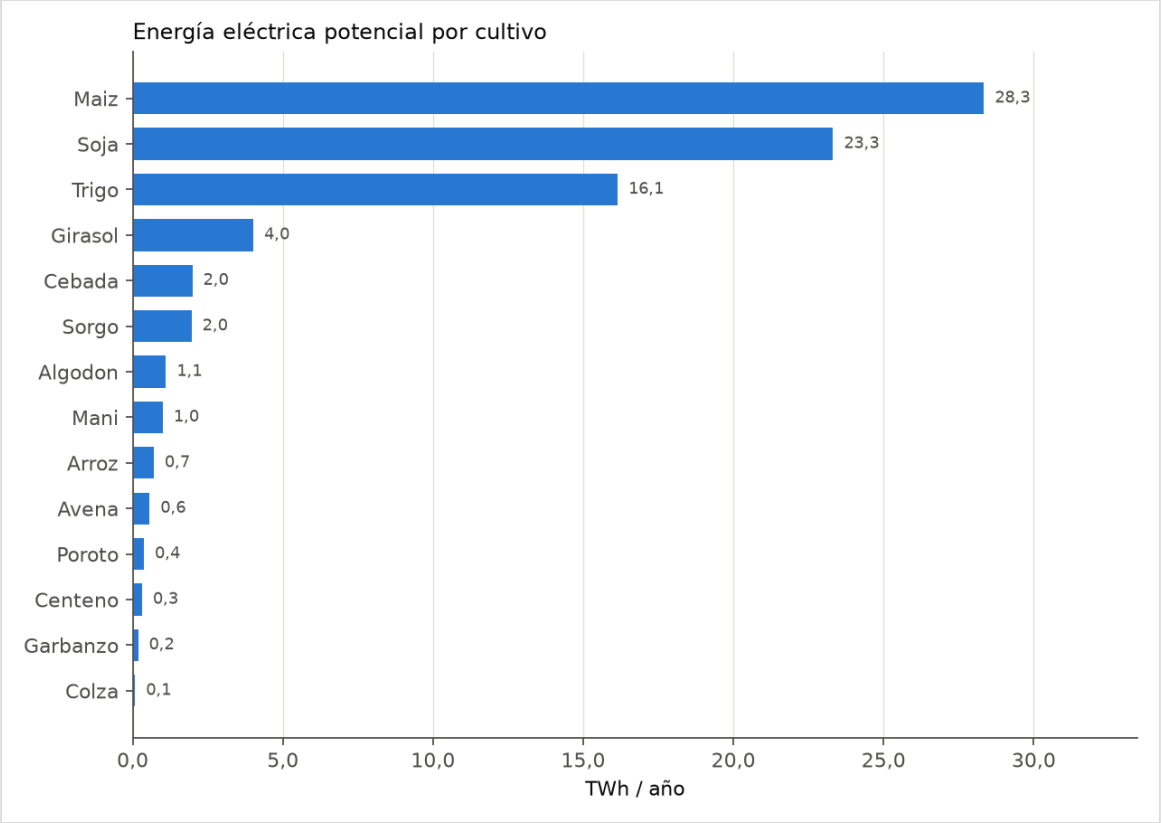


Figura 19. Energía eléctrica potencial por cultivo. Fuente: elaboración propia.

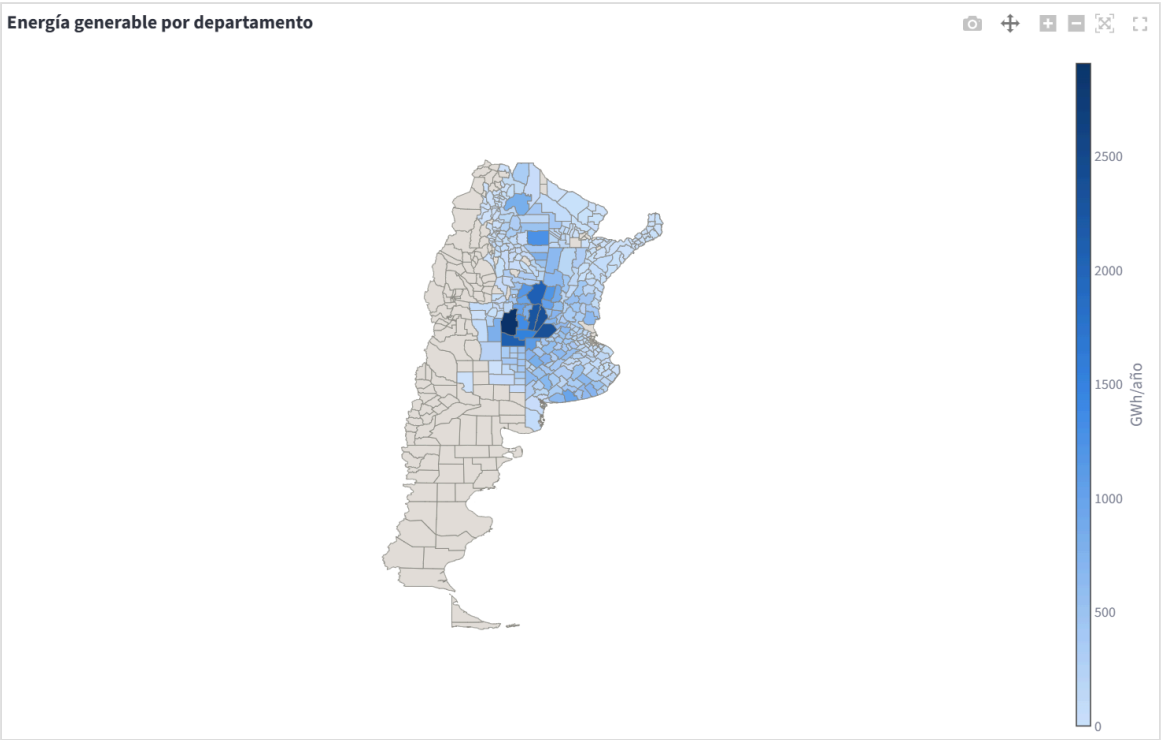


Figura 20. Mapa de energía generable por departamento (GWh/año). Fuente: captura del tablero.

10.5 Valorización económica

El beneficio bruto se distribuye de forma similar a la energía, aunque modulado por las tarifas: provincias con tarifas altas, como La Pampa o Salta, ganan participación económica respecto de su participación energética (Figuras 21 y 22). La tarifa de compra promedio ponderada por energía es de unos 150 $/kWh. Del beneficio total, el 64 % proviene del ahorro por autoconsumo y el 36 % del crédito por inyección.

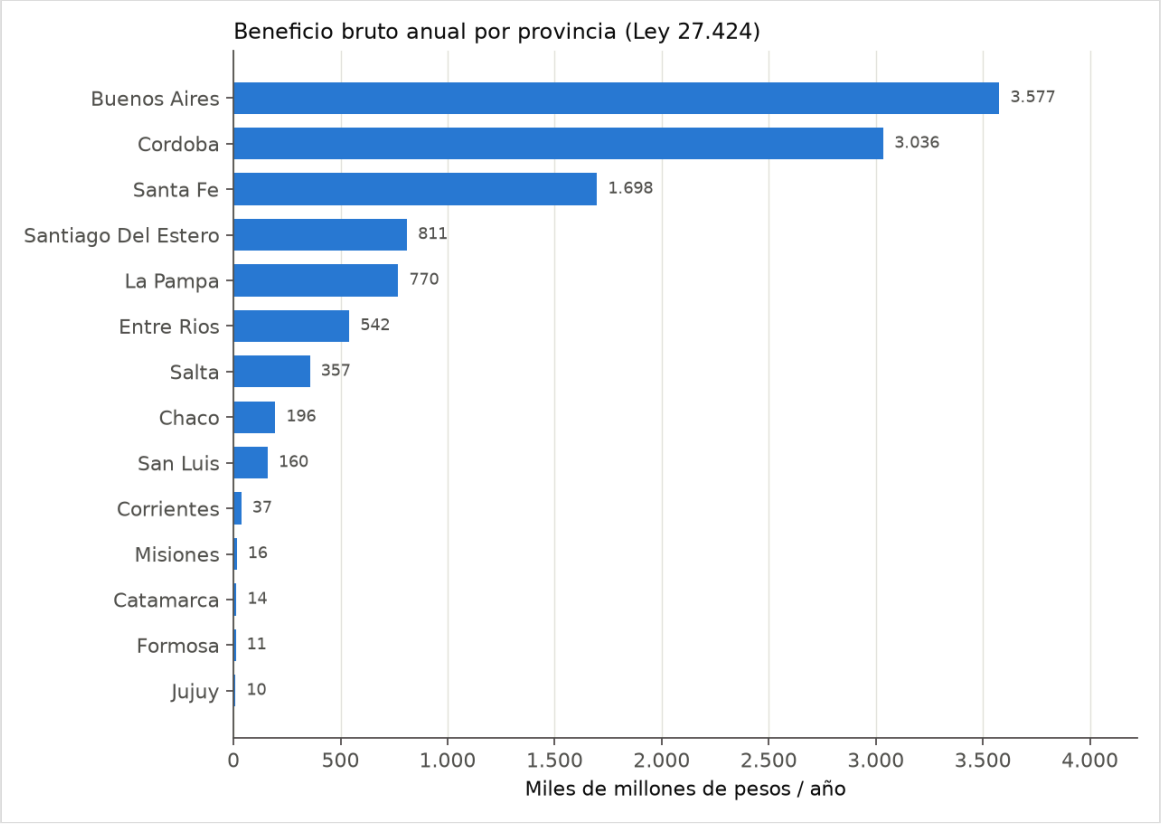


Figura 21. Beneficio bruto anual por provincia. Fuente: elaboración propia.

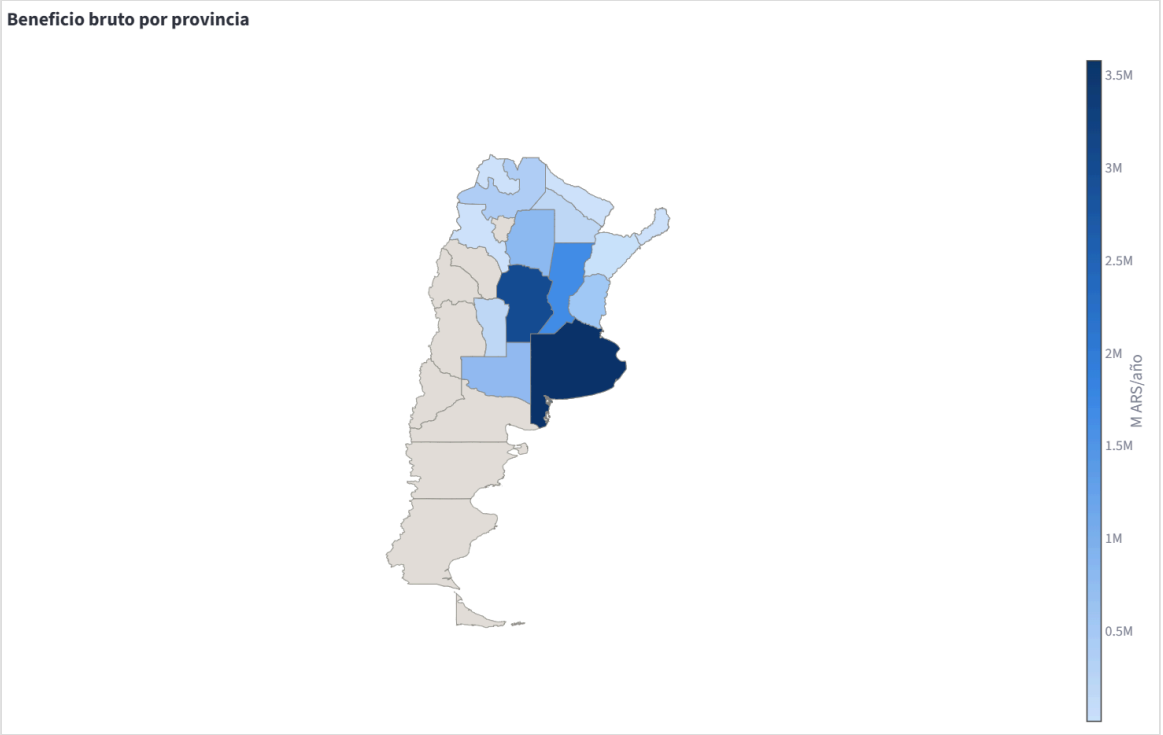


Figura 22. Mapa de beneficio bruto por provincia. Fuente: captura del tablero.

10.6 Análisis de sensibilidad

Se analizó la sensibilidad del beneficio a cada supuesto variándolo de a uno por vez (Saltelli et al., 2008). Como el modelo es lineal, la fracción retirada f afecta proporcionalmente a energía y dinero: entre 30 % y 70 %, el beneficio varía entre **6,7 y 15,7 billones de pesos por año** (Figura 23). El Informe de avance 2 cuantificó los demás supuestos sobre el período anterior: el escenario de inyección produce variaciones de entre −11 % y +1 %, el porcentaje de autoconsumo de entre −8 % y +5 %, y el consumo específico (1,0 frente a 1,1 kg MS/kWh) de hasta −10 %. La fracción retirada es, por amplio margen, **el supuesto más influyente**, y es también el más difícil de fijar con precisión, porque depende de las condiciones de cada suelo.

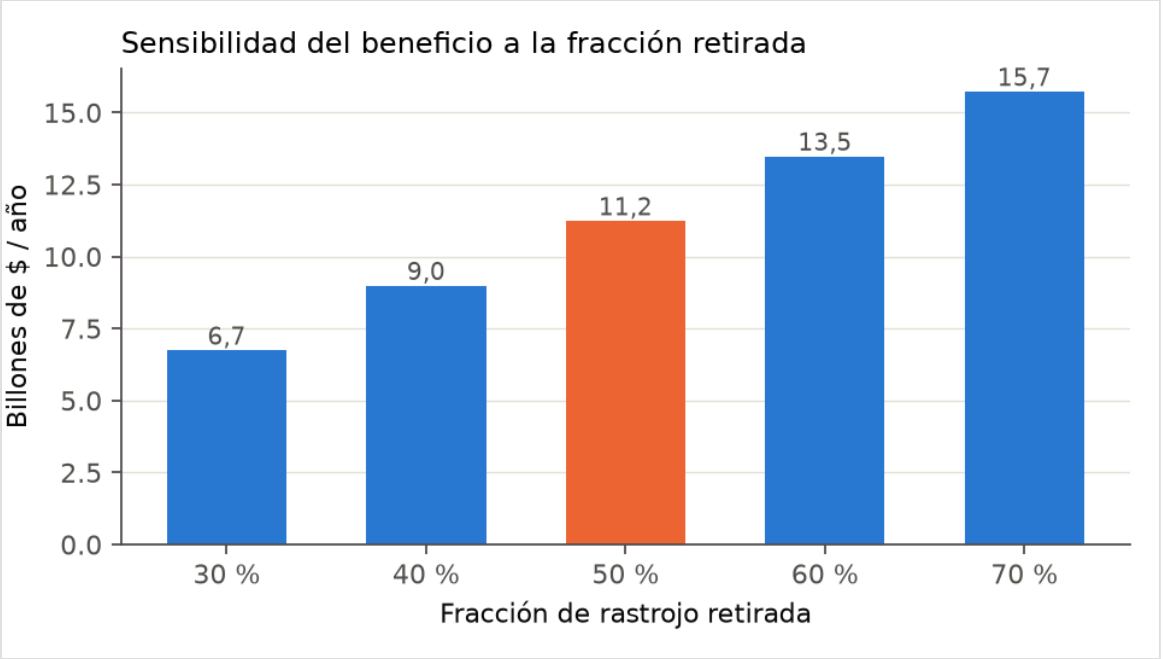


Figura 23. Sensibilidad del beneficio bruto a la fracción de rastrojo retirada. Fuente: elaboración propia.

10.7 Comparación con el Informe de avance 2

El Informe de avance 2 (octubre de 2026), con período 2020/21–2024/25, estimó 76,8 TWh/año y un beneficio de ≈10,5 billones de pesos (10,7 tras su fe de erratas). La diferencia con los valores actuales (80,1 TWh y 11,2 billones) se explica por el **desplazamiento del período de referencia**: al completarse la campaña 2025/26 salió del promedio la campaña 2020/21 y entró una campaña de mayor producción; además, se actualizaron las tarifas. Que el sistema recalcule estas cifras automáticamente, sin intervención manual, es precisamente uno de sus objetivos.

10.8 Conclusiones del capítulo

El potencial técnico de la biomasa residual agrícola argentina es significativo frente a la demanda eléctrica nacional y está fuertemente concentrado en el núcleo pampeano y en tres cultivos. El beneficio bruto asociado es elevado, pero debe interpretarse como una cota superior: no descuenta inversión, operación, acondicionamiento ni logística, y depende de manera sensible de la fracción de rastrojo que pueda retirarse sin dañar el suelo. Los resultados son útiles, sobre todo, para **comparar y priorizar territorios**, que es el uso para el que fue concebido un potencial técnico.

Capítulo 11. Conclusiones y trabajos futuros

11.1 Conclusiones generales

Este trabajo diseñó, construyó y puso en operación OPTIMIA-HV, un sistema de información que estima de manera automática el potencial eléctrico y económico de la biomasa residual agrícola de la Argentina a escala departamental. A continuación se evalúa el cumplimiento de cada objetivo específico.

1. **Estado del arte.** Se relevaron los fundamentos del dominio (índice de cosecha, fracción retirable, gasificación *downdraft*, balance neto) y de las tecnologías empleadas, incluidas las bases criptográficas y de protocolo de SSH (Capítulo 1).
2. **Extracción agrícola.** Se automatizó la obtención de las estimaciones del MAGyP y de la Bolsa de Cereales de Córdoba, con validación de formato, reintentos exponenciales, segunda pasada y respaldo en la última descarga (Capítulo 4).
3. **Extracción tarifaria.** Se automatizó la lectura de los cuadros de 25 de las 28 distribuidoras, con trazabilidad por SHA-256, determinación de vigencia, validación y un historial versionado; las tres restantes no publican documentos legibles por máquina (Capítulo 5).
4. **Modelo de cálculo.** Se implementó una cadena lineal y auditable de producción a biomasa, energía y dinero, con diez decisiones metodológicas registradas y justificadas (Capítulo 6).
5. **Calidad.** Se implementaron 156 casos de prueba, invariantes que impiden publicar datos imposibles, alertas comparativas y avisos por mensaje (Capítulo 7).
6. **Infraestructura.** El sistema se ejecuta en contenedores, se actualiza semanalmente de forma autónoma y se opera en un VPS con acceso web por HTTPS y acceso administrativo por SSH con clave pública, VPN y comandos forzados (Capítulo 8).
7. **Visualización.** Un tablero de seis páginas permite explorar los resultados, auditar los datos y modificar los supuestos más inciertos (Capítulo 9).

En cuanto al dominio, el sistema estima un potencial técnico de **80,1 TWh/año**, equivalente a cerca del 58 % de la demanda eléctrica nacional, y un beneficio bruto de **11,2 billones de pesos por año**, concentrados en el núcleo pampeano y en maíz, soja y trigo.

Más allá de las cifras, el trabajo deja tres aprendizajes de ingeniería. El primero es que, en sistemas que integran fuentes públicas, **el problema principal es la heterogeneidad y la fragilidad de las fuentes**, más que la complejidad del cálculo; por eso la mayor parte del código está dedicada a extraer y validar. El segundo es que **los errores más peligrosos son los silenciosos**: los que producen números verosímiles. La única defensa eficaz fue contrastar con las fuentes y convertir cada hallazgo en una prueba. El tercero es que **la**

trazabilidad y la explicitación de supuestos son requisitos de diseño, no documentación posterior: identificar cada decisión, conservar cada documento de origen y exponer lo descartado es lo que vuelve defendible un resultado.

11.2 Limitaciones

- El resultado es un **potencial técnico** con **beneficio bruto**: no incluye costos de inversión, operación, recolección, transporte ni acondicionamiento del rastrojo.
- Los coeficientes agronómicos son **constantes por cultivo** y no varían por región, rendimiento ni manejo.
- Se excluyen cultivos sin coeficientes publicados (caña de azúcar, yerba mate, cítricos), lo que **subrepresenta al NOA y al NEA**.
- La fracción retirable es **única** (50 %) y constituye una cota superior.
- El rastrojo suelto no es un combustible aprobado por el fabricante sin acondicionamiento previo.
- Tres distribuidoras (≈0,7 % de la energía) carecen de tarifa, y en una cuarta parte de la energía la tarifa de inyección es estimada.
- La asignación de tarifas usa la distribuidora provincial principal y no las cooperativas eléctricas, que abastecen buena parte de la ruralidad en varias provincias.

11.3 Mejoras a futuro

Modelo de dominio.

1. **Evaluación económica completa**: incorporar costos de inversión y operación del PP30, de recolección, enfardado y transporte, y calcular indicadores como el costo nivelado de la energía (LCOE), el valor actual neto y la tasa interna de retorno (Short et al., 1995; Sapag Chain et al., 2014; IRENA, 2012), para pasar de un potencial técnico a uno económico.
2. **Coeficientes regionalizados**: índices de cosecha y fracciones retirables por región, rendimiento y tipo de suelo, en lugar de valores nacionales.
3. **Nuevos cultivos y residuos**: incorporar caña de azúcar, yerba mate, cítricos y residuos agroindustriales (cáscaras, marlos), que son además los combustibles aprobados por el fabricante.
4. **Logística**: estimar distancias y costos de transporte para dimensionar plantas por radio de abastecimiento.
5. **Otras tecnologías**: comparar la gasificación con la combustión directa o la digestión anaeróbica, y con la integración solar que motivó la reestructuración explorada en octubre.
6. **Cooperativas eléctricas**: relevar las tarifas de las cooperativas, que son las que efectivamente abastecen a muchos usuarios rurales.

Sistema de información.

7. **Reconocimiento óptico de caracteres (OCR)** para leer los cuadros tarifarios escaneados de Tucumán y Tierra del Fuego.
8. **Despliegue continuo** condicionado a las pruebas, retomando lo aprendido en octubre: construcción y publicación de la imagen en un registro, despliegue por SSH con una clave restringida por comando forzado y verificación del chequeo de salud con reversión automática.
9. **Chequeos de salud y observabilidad**: agregar *healthchecks* a los servicios y exportar métricas y registros a una plataforma de monitoreo.
10. **Pruebas de contrato con las fuentes**: una ejecución periódica que verifique que los portales mantienen su estructura, para detectar cambios antes de que afecten una corrida.
11. **Autenticación del tablero**, si llegara a exponer información no pública.
12. **API de datos**: publicar los resultados mediante una API o archivos abiertos, para que otros sistemas los consuman.
13. **Actualización de dependencias** con un proceso automatizado y controlado por las pruebas.
14. **Sincronización de la documentación** con el código, mediante controles que detecten valores desactualizados (por ejemplo, el cociente de inyección o la frecuencia de ejecución).

11.4 Reflexión final

La transición energética no depende solo de nuevas tecnologías de generación, sino también de **información confiable y actualizada** para decidir dónde y cómo usarlas. Este trabajo muestra que, con herramientas abiertas y prácticas de ingeniería de software, es posible transformar datos públicos dispersos en un insumo de planificación que se mantiene vigente por sí solo. Esa es, en última instancia, la contribución de un ingeniero en sistemas de información a un problema energético: construir la infraestructura de datos que hace posible tomar mejores decisiones.

Referencias

Formato APA, 7.ª edición.

Bibliografía académica y técnica

- Aas, J., Barnes, R., Case, B., Durumeric, Z., Eckersley, P., Flores-López, A., Halderman, J. A., Hoffman-Andrews, J., Kasten, J., Rescorla, E., Schoen, S., y Warren, B. (2019). Let's Encrypt: An automated certificate authority to encrypt the entire web. En *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2473–2487). ACM. https://doi.org/10.1145/3319535.3363192
- Adamantiadis, A., Josefsson, S., y Baushke, M. (2020). *Secure Shell (SSH) key exchange method using Curve25519 and Curve448* (RFC 8731). IETF. https://doi.org/10.17487/RFC8731
- Armbrust, M., Ghodsi, A., Xin, R., y Zaharia, M. (2021). Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. En *11th Conference on Innovative Data Systems Research (CIDR)*.
- Barrett, D. J., Silverman, R. E., y Byrnes, R. G. (2005). *SSH, the Secure Shell: The definitive guide* (2.ª ed.). O'Reilly.
- Basu, P. (2018). *Biomass gasification, pyrolysis and torrefaction: Practical design and theory* (3.ª ed.). Academic Press.
- Batidzirai, B., Smeets, E. M. W., y Faaij, A. P. C. (2012). Harmonising bioenergy resource potentials—Methodological lessons from review of state of the art bioenergy potential assessments. *Renewable and Sustainable Energy Reviews, 16*(9), 6598–6630. https://doi.org/10.1016/j.rser.2012.09.002
- Bernstein, D. J. (2006). Curve25519: New Diffie-Hellman speed records. En *Public Key Cryptography – PKC 2006* (LNCS 3958, pp. 207–228). Springer.
- Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., y Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering, 2*(2), 77–89. https://doi.org/10.1007/s13389-012-0027-1
- Blanco-Canqui, H., y Lal, R. (2009). Crop residue removal impacts on soil productivity and environmental quality. *Critical Reviews in Plant Sciences, 28*(3), 139–163. https://doi.org/10.1080/07352680902776507
- Brooker, M. (2015, 4 de marzo). *Exponential backoff and jitter*. AWS Architecture Blog. https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book
- Christen, P. (2012). *Data matching: Concepts and techniques for record linkage, entity resolution, and duplicate detection*. Springer.
- Darghouth, N. R., Barbose, G., y Wiser, R. H. (2011). The impact of rate design and net metering on the bill savings from distributed PV for residential customers in California.

Energy Policy, 39(9), 5243–5253. https://doi.org/10.1016/j.enpol.2011.05.040
- Diffie, W., y Hellman, M. (1976). New directions in cryptography. *IEEE Transactions on Information Theory, 22*(6), 644–654. https://doi.org/10.1109/TIT.1976.1055638
- Donenfeld, J. A. (2017). WireGuard: Next generation kernel network tunnel. En *Network and Distributed System Security Symposium (NDSS)*. https://doi.org/10.14722/ndss.2017.23160
- Eco, U. (2001). *Cómo se hace una tesis: técnicas y procedimientos de estudio, investigación y escritura* (L. Baranda y A. Clavería, trads.). Gedisa. (Obra original publicada en 1977).
- Few, S. (2006). *Information dashboard design: The effective visual communication of data*. O'Reilly.
- Fielding, R., Nottingham, M., y Reschke, J. (2022). *HTTP semantics* (RFC 9110). IETF. https://doi.org/10.17487/RFC9110
- Fowler, M. (2006). *Continuous integration*. https://martinfowler.com/articles/continuousIntegration.html
- Harrower, M., y Brewer, C. A. (2003). ColorBrewer.org: An online tool for selecting colour schemes for maps. *The Cartographic Journal, 40*(1), 27–37. https://doi.org/10.1179/000870403235002042
- Hay, R. K. M. (1995). Harvest index: A review of its use in plant breeding and crop physiology. *Annals of Applied Biology, 126*(1), 197–216. https://doi.org/10.1111/j.1744-7348.1995.tb05015.x
- Hernández Sampieri, R., Fernández Collado, C., y Baptista Lucio, P. (2014). *Metodología de la investigación* (6.ª ed.). McGraw-Hill.
- Huber, P. J., y Ronchetti, E. M. (2009). *Robust statistics* (2.ª ed.). Wiley.
- Humble, J., y Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.
- Jenkins, B. M., Baxter, L. L., Miles, T. R., Jr., y Miles, T. R. (1998). Combustion properties of biomass. *Fuel Processing Technology, 54*(1–3), 17–46. https://doi.org/10.1016/S0378-3820(97)00059-3
- Kimball, R., y Caserta, J. (2004). *The data warehouse ETL toolkit*. Wiley.
- Lal, R. (2005). World crop residues production and implications of its use as a biofuel. *Environment International, 31*(4), 575–584. https://doi.org/10.1016/j.envint.2004.09.005
- Larman, C., y Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer, 36*(6), 47–56. https://doi.org/10.1109/MC.2003.1204375
- Lopez, A., Roberts, B., Heimiller, D., Blair, N., y Porro, G. (2012). *U.S. renewable energy technical potentials: A GIS-based analysis* (NREL/TP-6A20-51946). National Renewable Energy Laboratory.
- McKendry, P. (2002a). Energy production from biomass (part 2): Conversion technologies. *Bioresource Technology, 83*(1), 47–54. https://doi.org/10.1016/S0960-

8524(01)00119-5
- McKendry, P. (2002b). Energy production from biomass (part 3): Gasification technologies. *Bioresource Technology, 83*(1), 55–63. https://doi.org/10.1016/S0960-8524(01)00120-1
- McKinney, W. (2010). Data structures for statistical computing in Python. En *Proceedings of the 9th Python in Science Conference* (pp. 56–61). https://doi.org/10.25080/Majora-92bf1922-00a
- Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., y Vassilakis, T. (2010). Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment, 3*(1–2), 330–339.
- Menezes, A. J., van Oorschot, P. C., y Vanstone, S. A. (1996). *Handbook of applied cryptography*. CRC Press.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal, 2014*(239), 2.
- Mitchell, R. (2018). *Web scraping with Python: Collecting more data from the modern web* (2.ª ed.). O'Reilly.
- Myers, G. J., Sandler, C., y Badgett, T. (2011). *The art of software testing* (3.ª ed.). Wiley.
- Nottingham, M., y Fielding, R. (2012). *Additional HTTP status codes* (RFC 6585). IETF. https://doi.org/10.17487/RFC6585
- Nygard, M. (2011). *Documenting architecture decisions*. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions
- Raasveldt, M., y Mühleisen, H. (2019). DuckDB: An embeddable analytical database. En *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)* (pp. 1981–1984). ACM. https://doi.org/10.1145/3299869.3320212
- Reed, T. B., y Das, A. (1988). *Handbook of biomass downdraft gasifier engine systems* (SERI/SP-271-3022). Solar Energy Research Institute.
- Rescorla, E. (2018). *The Transport Layer Security (TLS) protocol version 1.3* (RFC 8446). IETF. https://doi.org/10.17487/RFC8446
- Rivest, R. L., Shamir, A., y Adleman, L. (1978). A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM, 21*(2), 120–126. https://doi.org/10.1145/359340.359342
- Saltelli, A., Ratto, M., Andres, T., Campolongo, F., Cariboni, J., Gatelli, D., Saisana, M., y Tarantola, S. (2008). *Global sensitivity analysis: The primer*. Wiley.
- Sapag Chain, N., Sapag Chain, R., y Sapag Puelma, J. M. (2014). *Preparación y evaluación de proyectos* (6.ª ed.). McGraw-Hill.
- Scarlat, N., Martinov, M., y Dallemand, J.-F. (2010). Assessment of the availability of agricultural crop residues in the European Union: Potential and limitations for bioenergy use. *Waste Management, 30*(10), 1889–1897. https://doi.org/10.1016/j.wasman.2010.04.016

- Short, W., Packey, D. J., y Holt, T. (1995). *A manual for the economic evaluation of energy efficiency and renewable energy technologies* (NREL/TP-462-5173). National Renewable Energy Laboratory.
- Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson.
- Stallings, W. (2017). *Cryptography and network security: Principles and practice* (7.ª ed.). Pearson.
- Tufte, E. R. (2001). *The visual display of quantitative information* (2.ª ed.). Graphics Press.
- Tumuluru, J. S., Wright, C. T., Hess, J. R., y Kenney, K. L. (2011). A review of biomass densification systems to develop uniform feedstock commodities for bioenergy application. *Biofuels, Bioproducts and Biorefining, 5*(6), 683–707. https://doi.org/10.1002/bbb.324
- Unkovich, M., Baldock, J., y Forbes, M. (2010). Variability in harvest index of grain crops and potential significance for carbon accounting: Examples from Australian agriculture. *Advances in Agronomy, 105*, 173–219. https://doi.org/10.1016/S0065-2113(10)05005-4
- Vassilev, S. V., Baxter, D., Andersen, L. K., y Vassileva, C. G. (2010). An overview of the chemical composition of biomass. *Fuel, 89*(5), 913–933. https://doi.org/10.1016/j.fuel.2009.10.022
- Wiggins, A. (2011). *The twelve-factor app*. https://12factor.net
- Wilhelm, W. W., Johnson, J. M. F., Hatfield, J. L., Voorhees, W. B., y Linden, D. R. (2004). Crop and soil productivity response to corn residue removal: A literature review. *Agronomy Journal, 96*(1), 1–17. https://doi.org/10.2134/agronj2004.1000
- Wilhelm, W. W., Johnson, J. M. F., Karlen, D. L., y Lightle, D. T. (2007). Corn stover to sustain soil organic carbon further constrains biomass supply. *Agronomy Journal, 99*(6), 1665–1667. https://doi.org/10.2134/agronj2007.0150
- Ylonen, T. (1996). SSH—Secure login connections over the Internet. En *Proceedings of the 6th USENIX Security Symposium* (pp. 37–42). USENIX.
- Ylonen, T., y Lonvick, C. (Eds.). (2006a). *The Secure Shell (SSH) protocol architecture* (RFC 4251). IETF. https://doi.org/10.17487/RFC4251
- Ylonen, T., y Lonvick, C. (Eds.). (2006b). *The Secure Shell (SSH) authentication protocol* (RFC 4252). IETF. https://doi.org/10.17487/RFC4252
- Ylonen, T., y Lonvick, C. (Eds.). (2006c). *The Secure Shell (SSH) transport layer protocol* (RFC 4253). IETF. https://doi.org/10.17487/RFC4253
- Ylonen, T., y Lonvick, C. (Eds.). (2006d). *The Secure Shell (SSH) connection protocol* (RFC 4254). IETF. https://doi.org/10.17487/RFC4254

Organismos, fabricantes y documentación de software

- All Power Labs. (s.f.-a). *PP30 one sheet* [ficha técnica]. https://www.allpowerlabs.com/wp-content/uploads/2019/08/PP30OneSheet2_1_23-2.pdf
- All Power Labs. (s.f.-b). *PP30 Power Pallet*. https://www.allpowerlabs.com/pp30-power-pallet
- Anthropic. (2026). *Claude Code* [software]. https://www.anthropic.com/claude-code
- Apache Software Foundation. (s.f.). *Apache Parquet documentation*. https://parquet.apache.org/docs/
- CAMMESA. (2023). *Informe anual 2022*. Compañía Administradora del Mercado Mayorista Eléctrico.
- Conventional Commits. (s.f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/es/v1.0.0/
- FAO. (2009). *Análisis del balance de energía derivada de biomasa en Argentina – WISDOM Argentina*. Organización de las Naciones Unidas para la Alimentación y la Agricultura. https://www.fao.org/4/i0900s/i0900s00.pdf
- FAO. (2020). *Actualización del balance de biomasa con fines energéticos en la Argentina – WISDOM Argentina* (Colección Documentos Técnicos N.º 19). FAO.
- GADM. (2022). *GADM database of global administrative areas, version 4.1*. https://gadm.org
- IRENA. (2012). *Renewable energy technologies: Cost analysis series. Volume 1: Power sector, Issue 1/5: Biomass for power generation*. International Renewable Energy Agency.
- Ministerio de Agricultura, Ganadería y Pesca. (s.f.). *Estimaciones agrícolas*. https://datosestimaciones.magyp.gob.ar
- The OpenBSD Project. (s.f.). *OpenSSH* y *PROTOCOL.certkeys*. https://www.openssh.com

Normativa

- Ley 27.424 de 2017. Régimen de Fomento a la Generación Distribuida de Energía Renovable Integrada a la Red Eléctrica Pública. *Boletín Oficial de la República Argentina*, 27 de diciembre de 2017.
- Decreto 986 de 2018. Reglamentación de la Ley 27.424. *Boletín Oficial de la República Argentina*.
- Resolución 314 de 2018 [Secretaría de Gobierno de Energía]. Normas de implementación de la Ley 27.424 (texto actualizado). https://www.argentina.gob.ar/normativa/nacional/norma-317996/actualizacion

Documentos del proyecto

- Ferro, M. (2026a). *Informe técnico de avance N.º 1 — OPTIMIA-HV* [informe no publicado]. `docs/informes/informe 1/`.
- Ferro, M. (2026b). *Informe técnico de avance N.º 2 — OPTIMIA-HV* [informe no publicado; fe de erratas del 27/10/2026]. `docs/informes/informe 2/`.
- Ferro, M. (2026c). *OPTIMIA-HV: código fuente y documentación técnica* [repositorio]. GitHub. `github.com/marcoeferro/SYNGAS-OPTIMIA-HV`.

Anexo A. Registro de decisiones metodológicas

Código	Decisión
POL-CROP-001	Si coexisten el total de un cultivo y sus componentes, prevalece el total; trigo candeal no se consolida con trigo.
POL-SOURCE-001	Entre MAGyP y BCCBA se conserva el registro de mayor producción; ante empate, MAGyP. Los descartados se auditan.
POL-MISSING-001	Un faltante («SD») no se convierte en cero; los registros no válidos se marcan, no se borran.
POL-GEO-001	Alias explícitos de departamentos entre fuentes, acotados por provincia.
POL-PERIOD-001	Resultados anuales como promedio de las 5 últimas campañas completas; ausencia = 0.
POL-ENERGY-001	1,0 kg MS/kWh y 22 kW a 50 Hz (ficha APL); la humedad (12 %) solo afecta la masa a cargar.
POL-RECOVERY-001	Fracción retirable del 50 % como cota superior sostenible; ajustable entre 20 % y 70 %.
POL-TARIFF-001	Tarifa de compra representativa: demandas medianas/grandes, BT, < 300 kW, banda resto, sin impuestos.
POL-TARIFF-002	Inyección faltante = compra × mediana del cociente observado; se marca como estimada.
POL-QA-001	Pruebas en CI; invariantes que detienen el ETL; alertas que avisan (y detienen en modo estricto).

Anexo B. Notación

Símbolo	Significado	Unidad / valor
P	Producción de grano	t
IC	Índice de cosecha	adimensional
FR	Factor residuo/producto = (1 − IC)/IC	adimensional
MS	Fracción de materia seca del producto	adimensional
f	Fracción de rastrojo retirable	0,5
CE	Consumo específico del PP30	1,0 kg MS/kWh
H	Humedad del material a cargar	12 %
P_{nom}	Potencia nominal del PP30	22 kW
D	Disponibilidad anual	0,8
α	Fracción autoconsumida	0,6
T_{compra}, T_{iny}	Tarifas de compra e inyección	$/kWh sin impuestos

Anexo C. Comandos de operación

Tarea	Comando
Levantar el sistema	`docker compose up -d --build`
Ejecutar el ETL manualmente	`docker compose --profile etl run --rm etl_pipeline`
Recalcular sin descargar	`python -m etl.pipelines.run_etl --skip-extract --skip-tarifas`
No publicar si hay alertas	`python -m etl.pipelines.run_etl --estricto`
Actualizar solo tarifas	`python -m etl.extractors.scrape_tarifas`
Ejecutar las pruebas	`pytest`
Abrir el tablero en desarrollo	`streamlit run dashboard/app.py`
Conectarse al servidor	`ssh -i ~/.ssh/id_ed25519 <usuario>@<servidor>`
Actualizar el servidor	`git pull && docker compose up -d --build`